

## CAPÍTULO 9

# Diagrama de Máquina de Estados

Esse diagrama era chamado de diagrama de gráfico de estados ou simplesmente diagrama de estados nas versões anteriores da UML. A partir da versão 2.0 da linguagem, seu nome foi modificado para diagrama de máquina de estados.

O diagrama de máquina de estados demonstra o comportamento de um elemento por meio de um conjunto finito de transições de estado. Além de ser utilizado para expressar o comportamento de uma parte do sistema, quando é chamado de máquina de estado comportamental, também pode ser usado para expressar o protocolo de uso de parte de um sistema, quando identifica uma máquina de estados de protocolo. Neste capítulo nos concentraremos em máquinas de estado comportamentais.

Uma máquina de estados comportamental pode ser usada para especificar o comportamento de vários elementos do modelo. O elemento modelado muitas vezes é uma instância de uma classe. No entanto, pode-se usar esse diagrama para modelar o comportamento de um caso de uso, por exemplo.

## 9.1 Estado

Um estado representa a situação em que um elemento (muitas vezes, um objeto) se encontra em determinado momento durante o período em que participa de um processo. Um objeto pode passar por diversos estados dentro de um mesmo processo. Um estado pode demonstrar:

- a espera pela ocorrência de um evento;
- a reação a um estímulo;
- a execução de alguma atividade;
- a satisfação de alguma condição.

### 9.1.1 Estado Simples

Um estado simples não tem subestados, ou seja, não pode ser subdividido em estados internos. É o tipo mais comum de estado. Por esse motivo, será referido simplesmente como estado ao longo do capítulo. A figura 9.1 apresenta um exemplo de estado simples.

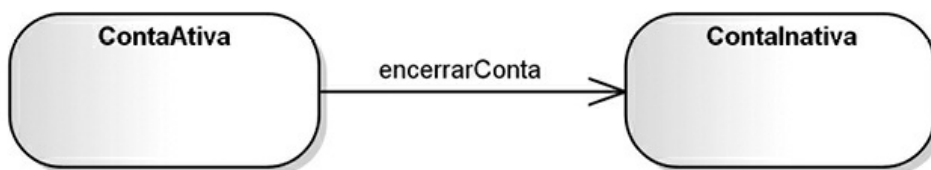


*Figura 9.1 – Exemplo de Estado Simples.*

O estado ilustrado nesse exemplo é um dos estados pelos quais passa um objeto da classe **ContaComum**. Aqui, o objeto se encontra no estado de **ContaAtiva**, ou seja, pode-se depositar valores nessa conta, bem como sacar valores, emitir saldos e extratos, além de outras operações.

## 9.2 Transições

Uma transição representa um evento que causa uma mudança no estado de um objeto, gerando um novo estado. Uma transição é representada por uma linha ligando dois estados, contendo uma seta em uma de suas extremidades, apontando para o novo estado gerado. A figura 9.2 apresenta um exemplo de transição entre estados.



*Figura 9.2 – Exemplo de Transição entre Estados.*

Nesse exemplo é apresentada uma transição que representa o evento de encerramento da conta. Aqui, existem dois estados: no primeiro, o objeto da classe **ContaComum** se encontra no estado ativo, enquanto, no segundo, o objeto se encontra no estado inativo. O evento que causou a mudança do estado do objeto é aqui representado por uma seta que parte do primeiro estado e atinge o segundo.

Uma transição pode ou não conter uma descrição (recomenda-se que tenha, para facilitar sua compreensão). A descrição de um evento pode

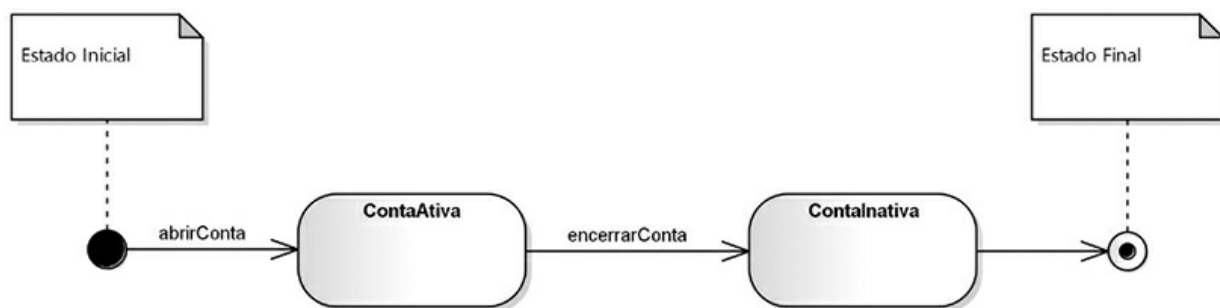
tanto conter uma ordem para realizar alguma tarefa como ser simplesmente uma informação avisando a ocorrência do evento. Além da descrição, uma transição pode também conter condições de guarda e parâmetros.

### 9.3 Estado Inicial

O estado inicial tem como função somente determinar o início da modelagem dos estados de um elemento. É representado por um círculo preenchido, a partir do qual é gerada uma transição que determina o início do processo. Da mesma forma que nas transições entre os objetos, uma transição de um estado inicial pode conter ou não uma descrição, o que, às vezes, é útil para identificar o evento que iniciou o processo.

### 9.4 Estado Final

O estado final tem a função apenas de indicar o final dos estados modelados. É representado por um círculo não preenchido envolvendo um segundo círculo preenchido. A figura 9.3 apresenta um exemplo de estados inicial e final.



*Figura 9.3 – Exemplo de Estados Inicial e Final – Processo de Emissão de Saldo.*

Nesse exemplo, usamos o componente nota para identificar os estados iniciais e finais desse diagrama. O diagrama em questão representa os estados pelos quais passa um objeto da classe **ContaComum**, que se resumem basicamente aos estados de contas ativa e inativa. O primeiro estado ocorre quando a conta é aberta, já o segundo ocorre quando a conta é encerrada.

Como o leitor pode perceber, os estados desse objeto são poucos. Isso é

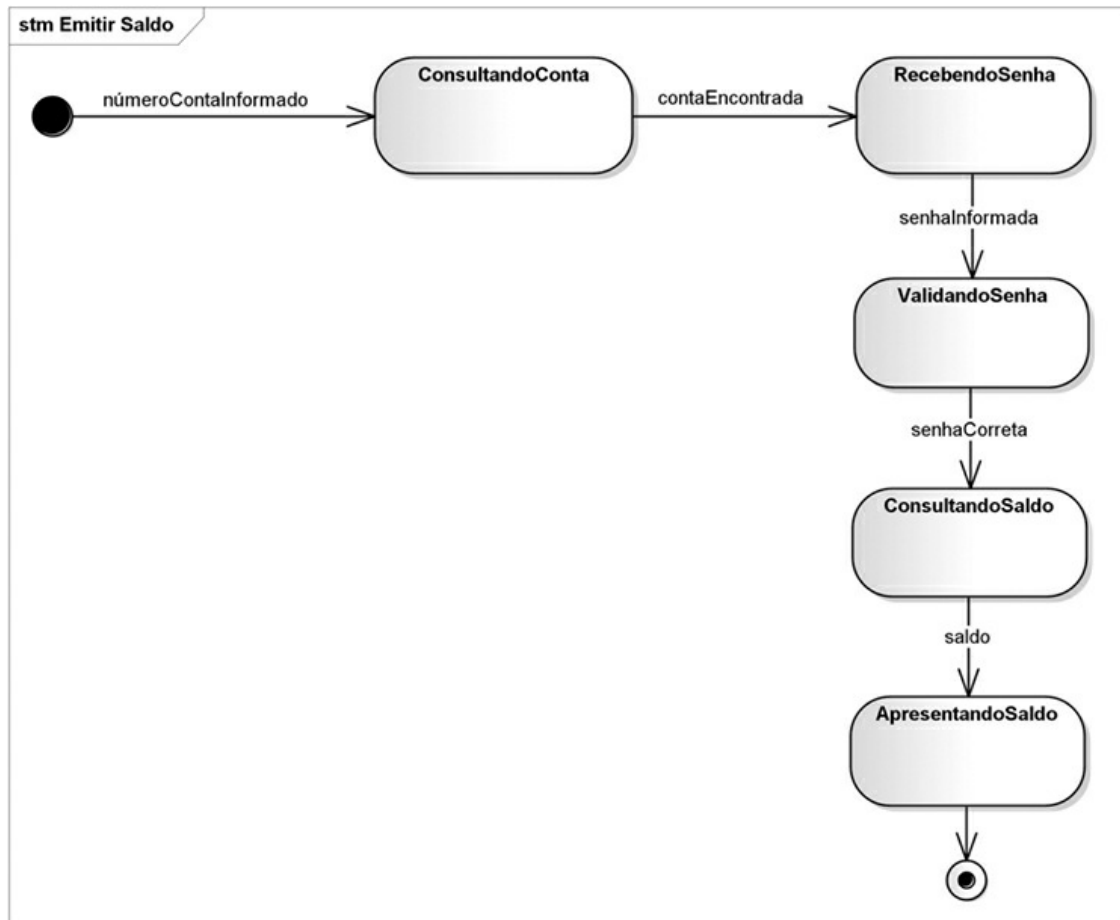
comum em sistemas de cunho comercial, em que os objetos não costumam possuir muitos estados. Em situações assim, é mais prático utilizar o diagrama de máquina de estados para modelar os estados de um processo representado por um caso de uso – que, nessa situação, passa a ser tratado como um objeto. Dessa forma, no restante deste capítulo, utilizaremos essa prática com frequência e muitos dos exemplos que apresentaremos serão relativos aos estados de um caso de uso.

## 9.5 Exemplo de Diagrama de Máquina de Estados – Processo de Emissão de Saldo

Aqui, tomaremos o caso de uso referente ao processo de emissão de saldo e representaremos os estados desse processo. Como já dissemos, em sistemas comerciais, essa prática é mais comum, posto que os objetos individuais costumam possuir poucos estados e estes muitas vezes são bastante simples (Figura 9.4).

Nesse exemplo é gerada uma transição a partir do estado inicial que dá início ao processo. Essa transição representa o evento em que um número de conta é informado, o que produz o estado **ConsultandoConta**.

Depois de a conta ter sido consultada, um novo evento é gerado, caracterizando-se pela informação de que a conta foi encontrada. Isto gera um novo estado, denominado **RecebendoSenha**, que é estático, ou seja, um estado em que não é realizada nenhuma atividade, apenas se aguarda que algo aconteça. Nesse estado específico, aguarda-se que a senha da conta seja informada.



*Figura 9.4 – Processo de Emissão de Saldo.*

No momento em que a senha é fornecida, é produzido o estado **ValidandoSenha**. Quando a validação for concluída, será solicitada a emissão do saldo, o que cria o estado **ConsultandoSaldo**. No momento em que esse estado é concluído, é gerada uma transição contendo o saldo a ser apresentado. Essa transição cria o estado **ApresentandoSaldo**, o que conclui esse processo, conforme demonstra o estado final.

É importante destacar que quando um estado está executando uma atividade, costuma ser descrito no gerúndio. No entanto, pode acontecer de um estado estar em uma situação estática, à espera de que um evento ocorra e, nessas situações, não deverá ser descrito no gerúndio.

## 9.6 Atividades internas

São as atividades que um objeto pode executar quando se encontra em um estado. O detalhamento delas não é obrigatório e nem sempre um estado as possuirá, mas elas fornecem informações extras sobre o que o objeto faz

quando se encontra em um estado, o que pode ser útil para compreender a função de um estado específico. Essas atividades podem ser detalhadas por meio das seguintes cláusulas:

- **Entry** – Identifica uma atividade executada quando o objeto assume (entra em) um estado. Sempre que um estado é assumido, executa o comportamento identificado por essa cláusula antes de qualquer outra ação.
- **Exit** – Identifica uma atividade executada quando o objeto sai de um estado. Sempre que se vai sair de um estado, o comportamento identificado por essa cláusula é executado antes de o estado ser abandonado.
- **Do** – Identifica uma atividade realizada durante o tempo em que o objeto se encontra em um estado. Atividades internas do tipo **Do** também são chamadas de atividades de estado.

As cláusulas **Entry** e **Exit**, também chamadas ações de estado, estão mais associadas às transições do que ao estado propriamente dito, já que são executadas quando o objeto assume um novo estado ou está mudando de estado. Além disso, seu tempo de execução costuma ser inferior ao das atividades de estado, podendo representar a simples atribuição de um valor a um atributo ou a geração de uma saída, enquanto as atividades de estado geralmente representam métodos executados pelo objeto.

As atividades internas são representadas em uma segunda divisão do estado, conforme demonstra a figura 9.5, na qual enriquecemos a figura 9.4 adicionando atividades internas aos estados, quando necessário.

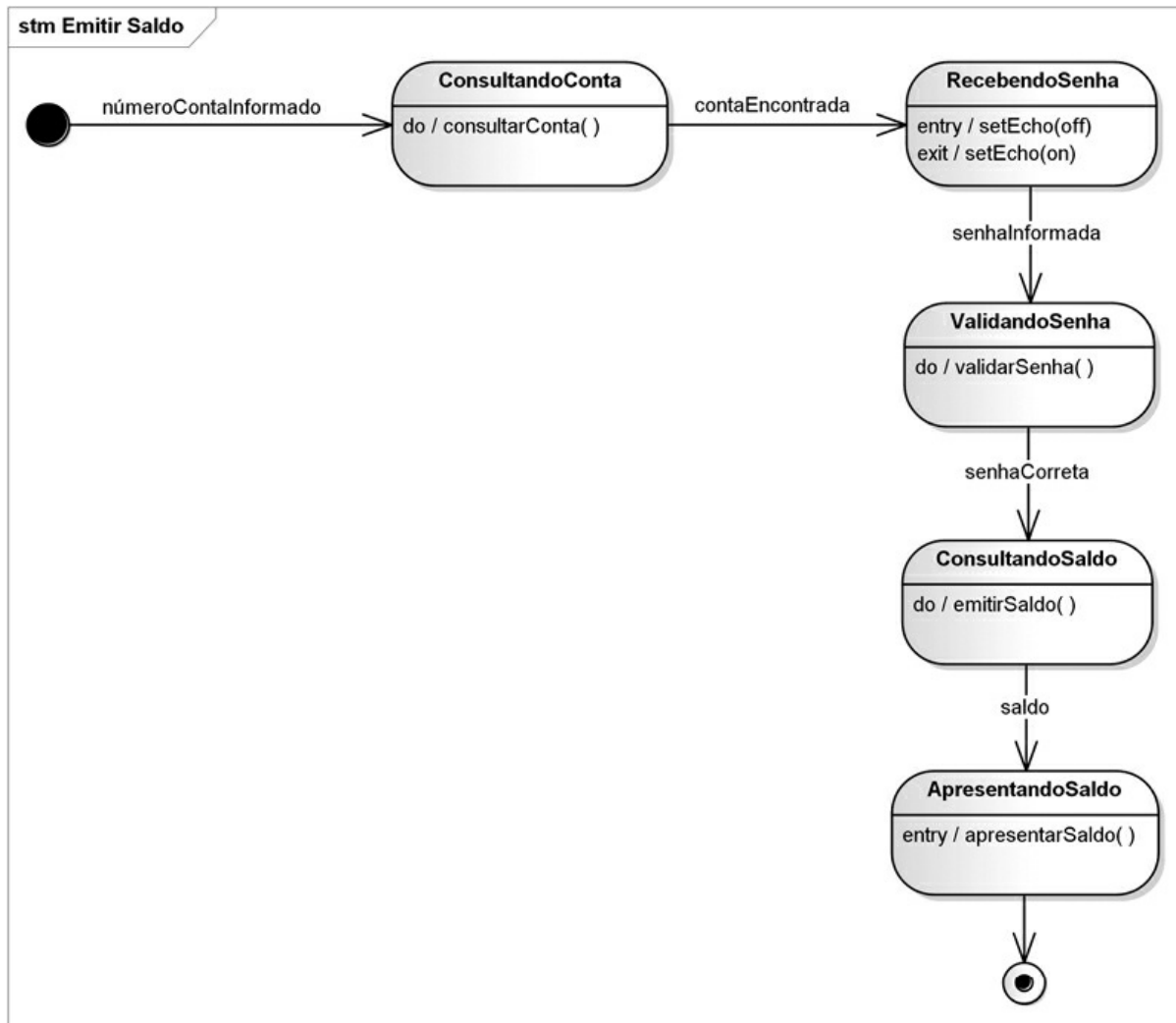


Figura 9.5 – Processo de Emissão de Saldo com Detalhamento de Atividades Internas.

Ao observar esse exemplo, o leitor notará que quando o objeto se encontra no estado **ConsultadoConta**, é executado o método **consultarConta**, conforme demonstra a cláusula **do**. Já quando o objeto entra no estado **RecebendoSenha**, é definido que qualquer caractere digitado não poderá ser mostrado, como é especificado pela cláusula **entry**, e quando o objeto sair desse estado, os caracteres passarão novamente a ser apresentados, conforme definido pela cláusula **exit**.

Da mesma forma, quando o objeto assumir o estado **ValidandoSenha**, será executado o método **validarSenha**, e quando o objeto estiver no estado **ConsultadoSaldo**, será executado o método **emitirSaldo**. Finalmente, quando o estado **ApresentandoSaldo** for atingido, o saldo

será apresentado, como é demonstrado pela cláusula **entry**.

## 9.7 Transições Internas

Transições internas não produzem modificações no estado de um objeto. A figura 9.6 apresenta um exemplo de transição interna.



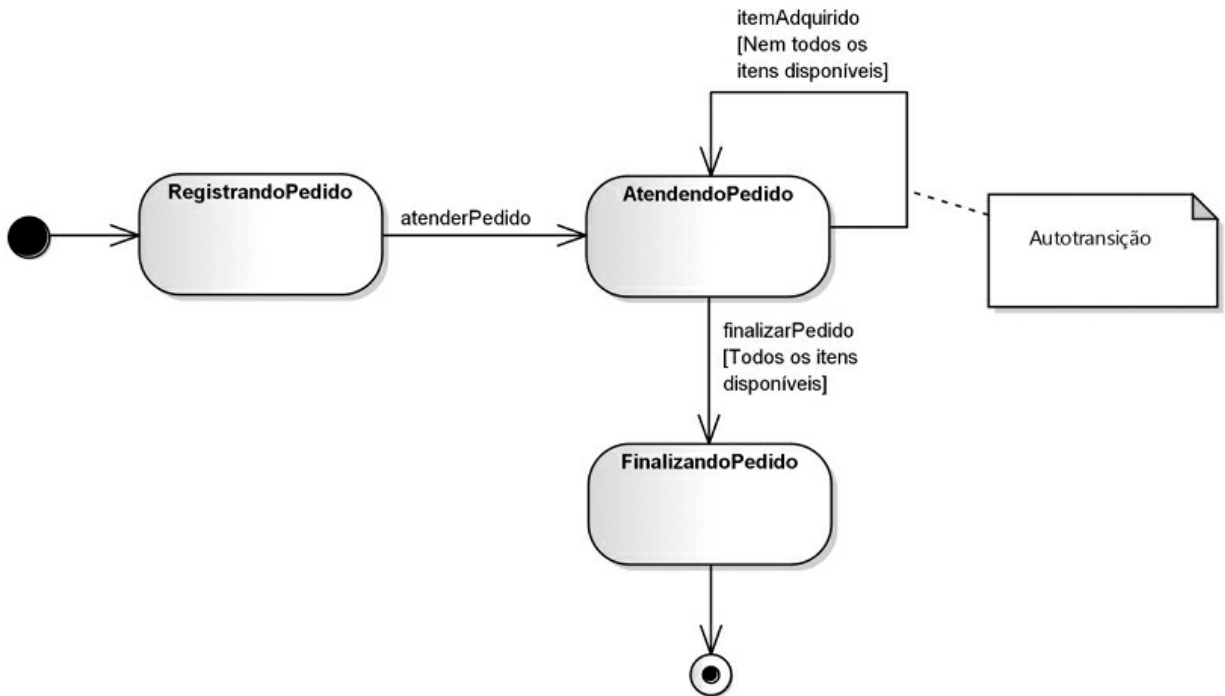
*Figura 9.6 – Exemplo de Transição Interna.*

Nesse exemplo, enfocamos o estado **RegistrandoPessoa**, cuja função é cadastrar um novo cliente da instituição bancária, representado por uma pessoa física. Nesse estado, existem duas cláusulas: a primeira representa uma atividade de estado, conforme demonstra a cláusula **Do**, que determina que deverá ser executado o método **registrarPessoa** quando o objeto se encontrar nesse estado. A segunda representa uma transição interna que determina que o CPF da pessoa deverá ser validado por meio da execução do método **validarCpf**, que será chamado durante a execução do método **registrarPessoa**, ou seja, antes de concluir o registro da pessoa, é necessário verificar se o CPF informado está correto. Assim, embora o objeto esteja validando o CPF informado pelo cliente, ainda se encontra no estado **RegistrandoPessoa**, não havendo uma mudança no estado do objeto.

## 9.8 Autotransições

Autotransições apresentam pequenas diferenças em relação às transições internas. Essas últimas ocorrem durante um estado do objeto, sem modificá-lo, enquanto as autotransições saem do estado atual do objeto, podendo executar alguma ação quando dessa saída, e retornam ao mesmo estado. Uma autotransição é representada por uma seta de transição que parte do objeto e retorna ao próprio objeto. A figura 9.7 apresenta um exemplo de autotransição.





*Figura 9.7 – Exemplo de Autotransição.*

Nesse exemplo, enfocamos o processo de atendimento de um pedido. Pode acontecer de, ao atender a um pedido, nem todos os itens solicitados estarem disponíveis em estoque, sendo preciso adquiri-los. Assim, sempre que um novo item for adquirido, ocorrerá uma autotransição. No entanto, se algum item ainda estiver indisponível, o objeto voltará ao estado **Atendendo Pedido**. Somente quando todos os itens do pedido estiverem disponíveis, será possível passar ao estado seguinte.

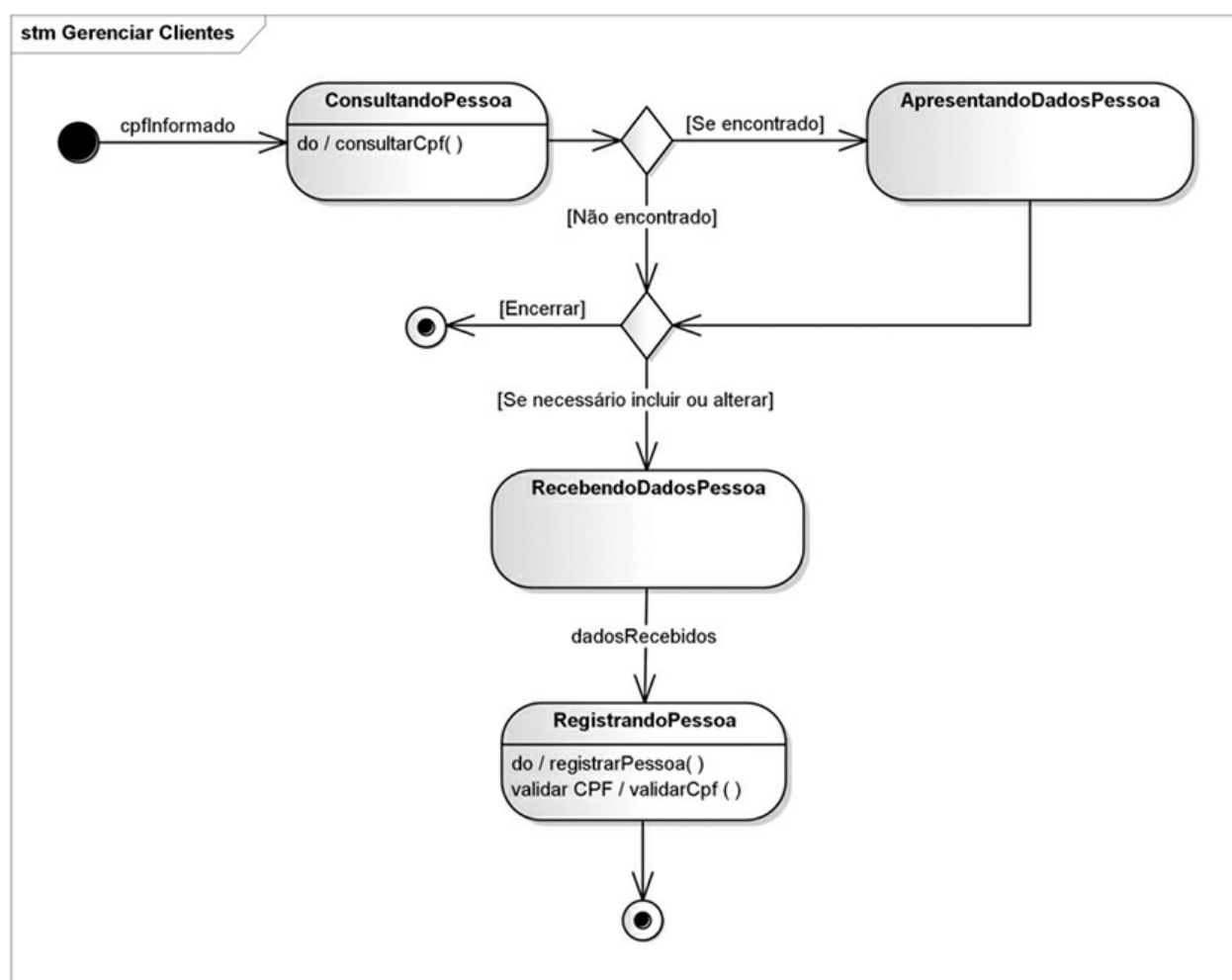
Observe que existem duas condições de guarda associadas às transições que determinam a condição para que se permaneça no estado **AtendendoPedido** ou se passe para o estado **FinalizandoPedido**. O evento que causa a autotransição é identificado pela aquisição de um novo item. A esse evento existe uma condição de guarda associada, definida como “Nem todos os itens disponíveis”, o que determina que se deve voltar ao mesmo estado quando essa condição for verdadeira. Já a transição que leva ao novo estado **FinalizandoPedido** tem como condição que todos os itens estejam disponíveis.

## 9.9 Pseudoestado de Escolha

Conhecido nas versões anteriores como estado de ponto de escolha

dinâmico, o pseudoestado de escolha representa um ponto na transição de estados de um objeto em que deve ser tomada uma decisão, segundo a qual um determinado estado será ou não gerado, normalmente em detrimento de diversos outros possíveis estados. Dessa forma, um pseudoestado de escolha representa uma decisão, apoiada por condições de guarda, em que se decidirá qual o próximo estado do objeto a ser gerado.

Um pseudoestado de escolha pode ser representado por um losango ou um círculo vazio de onde partem duas ou mais possíveis transições. A figura 9.8 apresenta um exemplo de pseudoestado de escolha enfocando o processo de gerenciamento do cadastro de clientes do sistema de controle bancário.



*Figura 9.8 – Pseudoestado de Escolha – Gerenciar Clientes.*

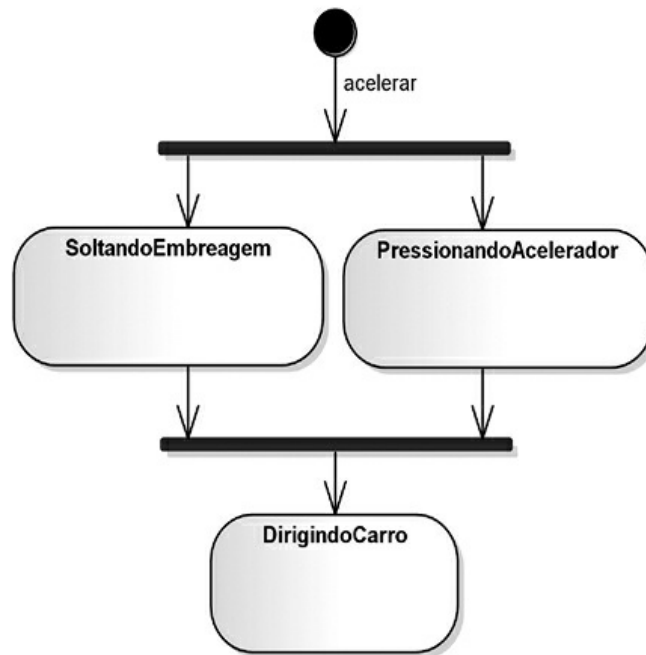
Ao observarmos a máquina de estados apresentada no diagrama, percebemos que o processo se inicia quando é informado o CPF da pessoa,

o que gera o estado **ConsultandoPessoa**, em que é executado o método **consultarCpf**. Quando o estado é concluído, é necessário fazer uma escolha, representada por um pseudoestado de escolha, em que, se for encontrada uma pessoa com o CPF informado, o processo deverá passar ao estado **ApresentandoDadosPessoa** (conforme demonstra a condição de guarda) e, em seguida, a um segundo pseudoestado de escolha.

Se a pessoa não for encontrada, então o estado **ApresentandoDadosPessoa** não será gerado, passando-se diretamente ao segundo pseudoestado de escolha, em que se deve escolher encerrar o processo ou fornecer os dados da pessoa para inserir uma nova pessoa ou alterar os dados da pessoa encontrada, o que produz o estado **RecebendoDadosPessoa** e, no momento em que este for concluído, passa-se ao estado **RegistrandoPessoa**, no qual serão executados os métodos **registrarPessoa** e **validarCpf**, já explicados.

## 9.10 Barra de Bifurcação/União

É utilizada quando da ocorrência de estados paralelos, causados por transições concorrentes. Sua função é determinar o momento em que o processo passou a ser executado em paralelo e em quantos subprocessos se dividiu (evento conhecido como bifurcação) ou determinar o momento em que dois ou mais subprocessos se uniram em um único processo (evento conhecido como união). Pode ser representada tanto por uma barra horizontal quanto vertical. A figura 9.9 apresenta um exemplo de barra de bifurcação/união, em que são modelados dois estados paralelos pelos quais passa um objeto da classe carro no momento em que um ator necessita dirigi-lo.

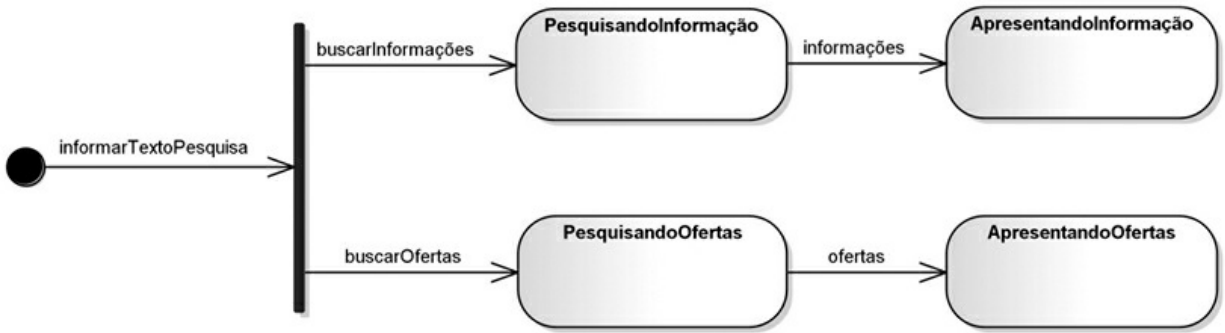


*Figura 9.9 – Barra de Bifurcação/União.*

Esse exemplo representa um trecho de um diagrama que modela os estados pelos quais passa um objeto da classe **Carro** no processo que o usuário leva para destrancá-lo, ligá-lo e dirigi-lo. No trecho ilustrado na figura, enfocamos o momento final do processo, quando o motorista começa a guiar o carro, mas, para isso, ele precisa realizar duas tarefas simultaneamente: soltar a embreagem e pressionar o acelerador. Dessa forma, foi inserida uma barra de bifurcação/união horizontal, indicando que os dois estados ocorrem em paralelo. Em seguida, usa-se uma nova barra de bifurcação/união para unir os dois subprocessos e gerar o estado **DirigindoCarro**.

Outro exemplo para o uso da barra de bifurcação/união pode ser visto na figura 9.10, em que enfocamos um processo que se refere a uma pesquisa de internet, na qual, em paralelo com a busca por informações solicitadas, são pesquisadas também ofertas relacionadas à pesquisa.

Nesse exemplo, no momento em que o evento **informarTextoPesquisa** é disparado, uma barra de bifurcação/união divide os estados gerados pelo evento, havendo um fluxo para buscar e apresentar as informações pertinentes à pesquisa e outro para buscar e apresentar as ofertas relacionadas a esta.



*Figura 9.10 – Uso da barra de bifurcação/união no processo de Solicitar Pesquisa.*

## 9.11 Estados Compostos

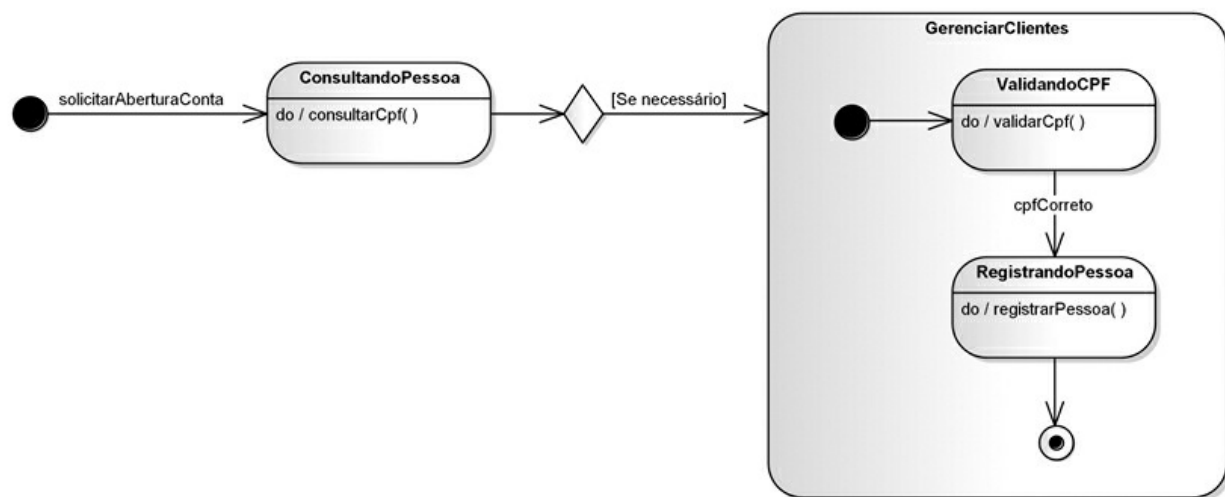
Estados compostos são o segundo tipo de estado suportado pelo diagrama de máquina de estados. O primeiro é o estado simples, explicado anteriormente neste capítulo, e o terceiro é a máquina de subestados, que será explicada mais adiante.

Um estado composto contém internamente dois ou mais estados chamados subestados. Um subestado é chamado direto, quando não é contido por outro estado, ou indireto, caso contrário. Assim, estado composto é um estado que foi “explorado”, de maneira a apresentar detalhadamente todas as etapas pelas quais passa o objeto quando no estado em questão.

Um estado composto pode apresentar somente uma região ou ser decomposto em duas ou mais regiões ortogonais, quando se torna um estado ortogonal (chamado estado concorrente em versões anteriores), em que cada região terá estados e transições. Cada região de um estado composto pode ter um pseudoestado inicial e um estado final. Uma transição para um estado composto representa uma transição para o pseudoestado inicial de cada região. Um estado composto não pode ser reutilizado, diferentemente do que ocorre com as máquinas de subestado.

Um estado composto pode facilitar a compreensão de um determinado estado de maneira bem mais detalhada, em que são discriminados os diversos subestados pelos quais passa o objeto quando no estado composto. A figura 9.11 demonstra um exemplo de estado composto com apenas uma região.

No exemplo dessa figura, iniciamos a modelar o processo de abertura de conta no qual o evento **solicitarAberturaConta** produz uma transição que gera o estado **ConsultandoPessoa**. Depois de esse estado ser concluído, passa-se a um pseudoestado de escolha (somente identificamos uma de suas transições, uma vez que o exemplo está incompleto), no qual, se for necessário (como demonstra a condição de guarda), gera-se uma transição para um estado composto intitulado **GerenciarClientes**, que representa os estados necessários para dar manutenção no cadastro de pessoas, ou seja, para incluir ou alterar o registro de uma pessoa.



*Figura 9.11 – Estado Composto.*

O leitor notará que transformamos o estado **RegistrandoPessoa**, exemplificado anteriormente, em um estado composto, dividindo-o em dois subestados. Nesse caso, tomamos a transição interna para validação de CPF e a transformamos em um subestado, assim como criamos um segundo subestado, denominado **RegistrandoPessoa**. Dessa forma, antes de concluir o cadastro ou a atualização da pessoa, primeiramente se valida seu CPF e, caso este esteja realmente correto, registram-se os dados da pessoa.

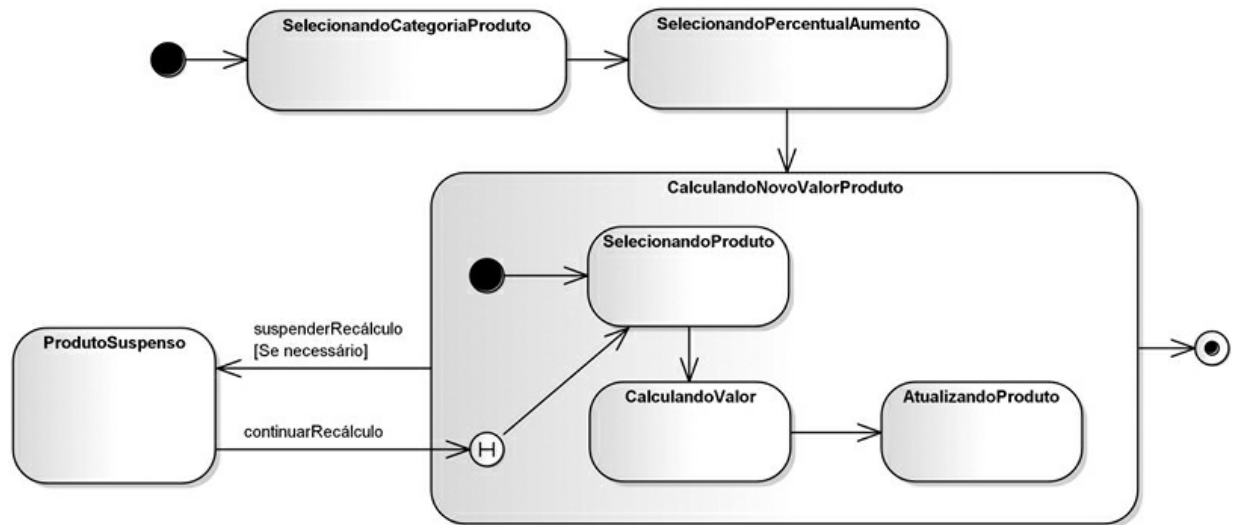
O estado inicial apresentado dentro do estado composto é um pseudoestado inicial, já que o diagrama tem um estado inicial. Esse pseudoestado inicial refere-se somente ao início dos subestados do estado composto.

## 9.12 Pseudoestado de História

Um pseudoestado de história representa o registro do último subestado em que um objeto se encontrava, quando, por algum motivo, o processo foi interrompido. Assim, por meio do pseudoestado de história, podemos retornar exatamente ao último subestado em que o objeto encontrava-se quando da interrupção do processo. Um pseudoestado de história é representado por um **H** dentro de um círculo, conforme ilustra a figura 9.12.

Nesse exemplo, enfocamos um processo de aumento de produtos de uma determinada categoria. Primeiramente, selecionamos a categoria do produto e, depois, o percentual de aumento para os produtos da categoria escolhida. O recálculo de valor de todos os produtos da categoria em questão é representado por um estado composto, em que um produto específico da categoria é selecionado, seu valor de venda é recalculado e o produto, atualizado. Observe que um estado de história mantém o subestado atual do processo.

Eventualmente, por um motivo qualquer, pode ser necessário suspender temporariamente o processo, conforme demonstra a transição à esquerda do estado composto, que gera o estado **ProdutoSuspenso**. Tão logo o motivo da interrupção seja sanado, o processo de recálculo poderá ser retomado a partir do momento em que foi suspenso, por meio do estado de história. Observe que é gerada uma transição do estado **ProdutoSuspenso** diretamente para o estado de história, que será responsável por continuar o processo exatamente a partir do subestado em que havia sido interrompido.



*Figura 9.12 – Pseudoestado de História.*

Existe também o pseudoestado de história profunda, representado por um **H** seguido de um símbolo de asterisco (**H\***) envolvido por um círculo, utilizado quando da ocorrência de estados compostos dentro de estados compostos. Assim, o estado de história profunda guarda e recupera o último subestado em qualquer dos estados compostos onde possa estar.

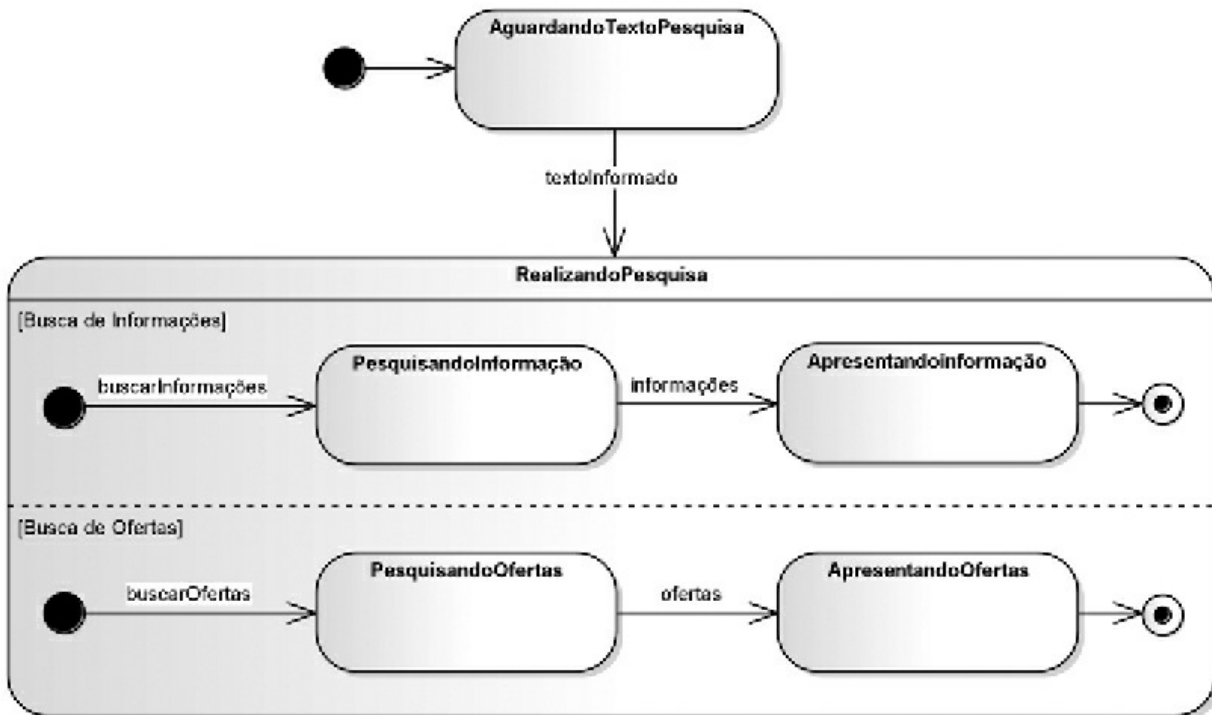
### 9.13 Estados Compostos Ortogonais

Um estado ortogonal (chamado concorrente em versões anteriores) é um estado composto que possui mais de uma região, onde cada uma apresenta um conjunto de estados e os estados de cada região são assumidos paralelamente, o que força o processo a se dividir em dois ou mais subprocessos concorrentes. Os processos concorrentes são separados por uma linha tracejada dentro do estado ortogonal, conforme pode ser observado na figura 9.13.

Esta figura apresenta uma segunda alternativa para o processo de **Solicitar Pesquisa**. Nesse exemplo, o processo se inicia com um estado estático que aguarda que o usuário digite um texto a pesquisar. O evento de inserção de um texto para pesquisa causa uma transição para um estado composto ortogonal contendo duas regiões. Na primeira região, como seu título na forma de condição de guarda informa, é feita a busca por informações relevantes à pesquisa, enquanto na segunda região é feita a busca por ofertas relacionadas ao texto pesquisado. Os estados de cada



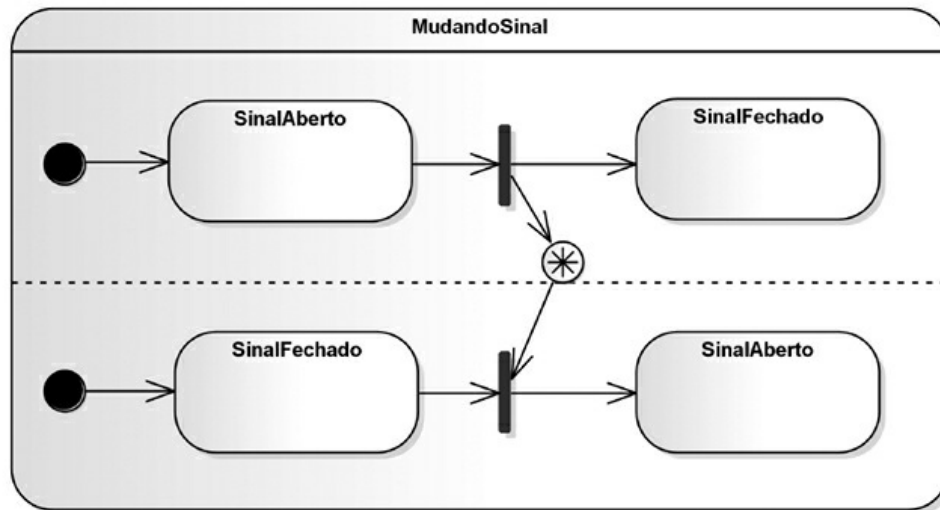
região ocorrem paralelamente.



*Figura 9.13 – Exemplo de Estado Composto Ortogonal.*

## 9.14 Estado de Sincronismo

Em alguns processos pode eventualmente ser necessário que estados de regiões diferentes estejam de alguma forma sincronizados, por vezes sendo necessário que um estado de uma região espere por um estado de outra. Assim, é preciso existir um estado de sincronismo cuja função é permitir que os relógios de duas ou mais regiões estejam sincronizados em um determinado momento do processo. O estado de sincronismo é representado por um símbolo de asterisco (\*) dentro de um círculo, conforme pode ser observado na figura 9.14.

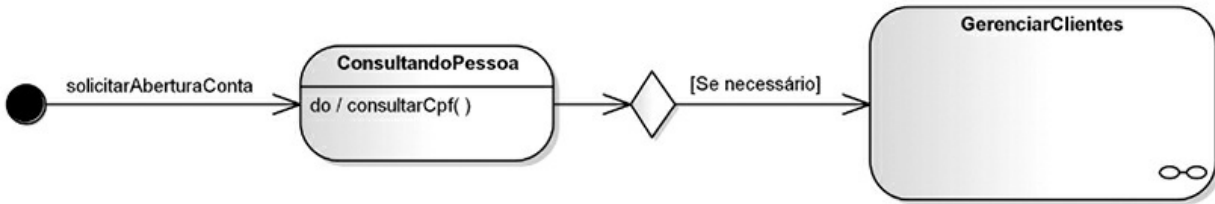


*Figura 9.14 – Estado de Sincronismo.*

Nesse exemplo, há dois sinais de trânsito. No momento em que o sinal de um muda, o outro deve mudar também, automaticamente. Assim, o estado do primeiro sinal de trânsito é representado como **SinalAberto**, e o do segundo, como **SinalFechado**. Quando o primeiro sinal de trânsito recebe a ordem de trocar seu sinal, este gera duas transições, uma para trocar seu próprio estado e outra para avisar o estado de sincronismo para alterar também o estado do segundo sinal. Assim, no momento em que o estado do primeiro sinal mudar para **SinalFechado**, o estado do segundo sinal automaticamente terá de mudar para **SinalAberto**.

## 9.15 Estado de Submáquina

Um estado de submáquina é um mecanismo de decomposição que permite a fatoração de comportamentos comuns e seu reuso. Um estado de submáquina é equivalente a um estado composto. No entanto, seus subestados não são descritos no diagrama, o que indica que terão de ser demonstrados em outro diagrama. Além disso, diferentemente de um estado de submáquina, um estado composto não pode ser reutilizado. Um estado de submáquina é representado por um retângulo com as bordas arredondadas sem divisões internas, contendo no canto inferior direito um símbolo que representa um diagrama de máquina de estados, significando que o estado em questão possui subestados internos. A figura 9.15 apresenta um exemplo de estado de submáquina.



*Figura 9.15 – Estado de Submáquina.*

Aqui, utilizamos novamente o exemplo parcial do processo de **Abertura de Conta** já apresentado e substituímos o estado composto denominado **Gerenciar Clientes** por um estado de submáquina de mesmo nome, em que os subestados não são representados, mas deverão estar detalhados em outro diagrama de máquina de estados, normalmente com o nome do próprio estado de submáquina. Essa abordagem permite produzir diagramas mais enxutos, deixando os diagramas mais fáceis de entender, além de permitir referenciar outros diagramas já modelados.

## 9.16 Pseudoestado de Junção

Esse pseudoestado é utilizado para projetar caminhos transacionais complexos. Pode unir múltiplos fluxos em um único ou dividir um fluxo em diversos, sendo possível utilizar condições de guarda como auxílio. Um pseudoestado de junção é representado por um símbolo idêntico ao de estado inicial, ou seja, um círculo preenchido. A figura 9.16 apresenta um exemplo de pseudoestado de junção utilizado durante o processo de abertura de conta.

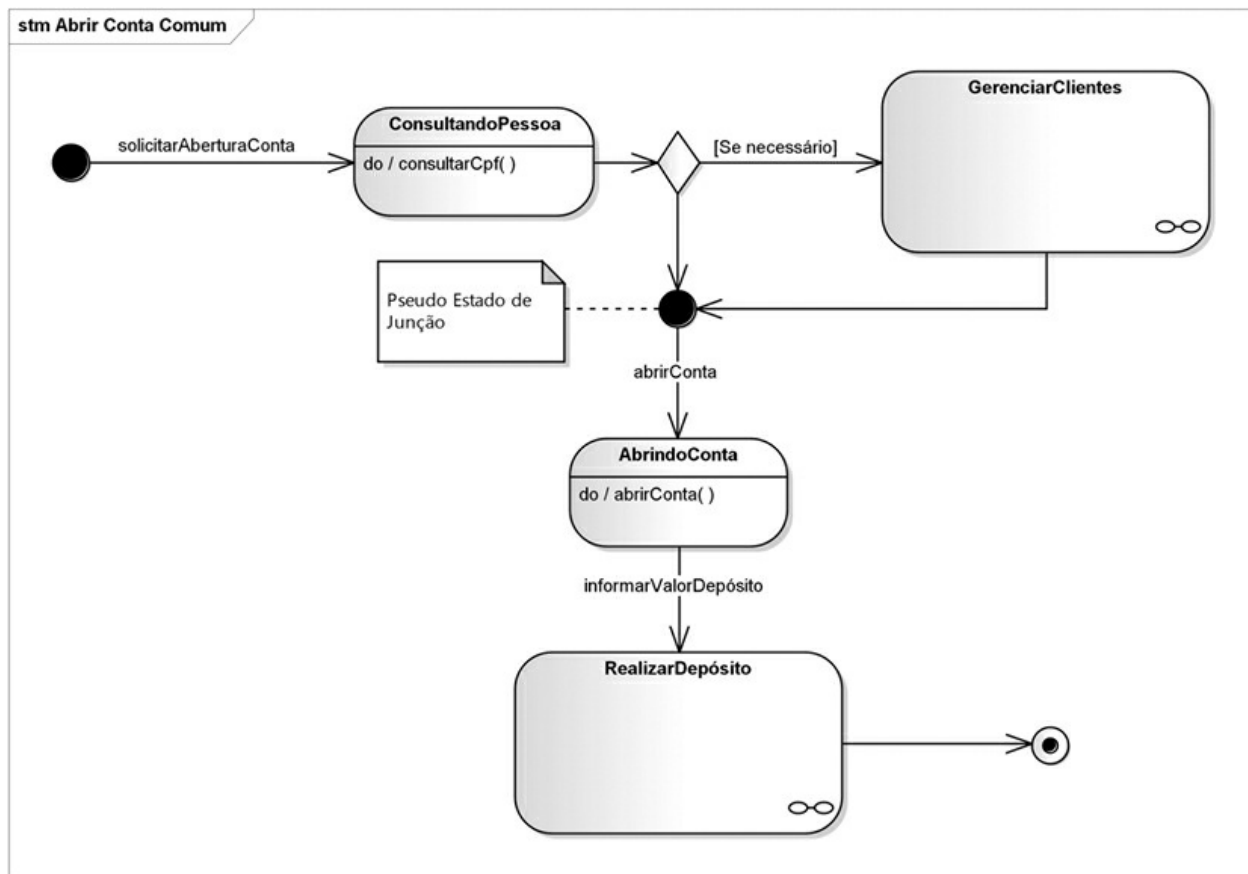


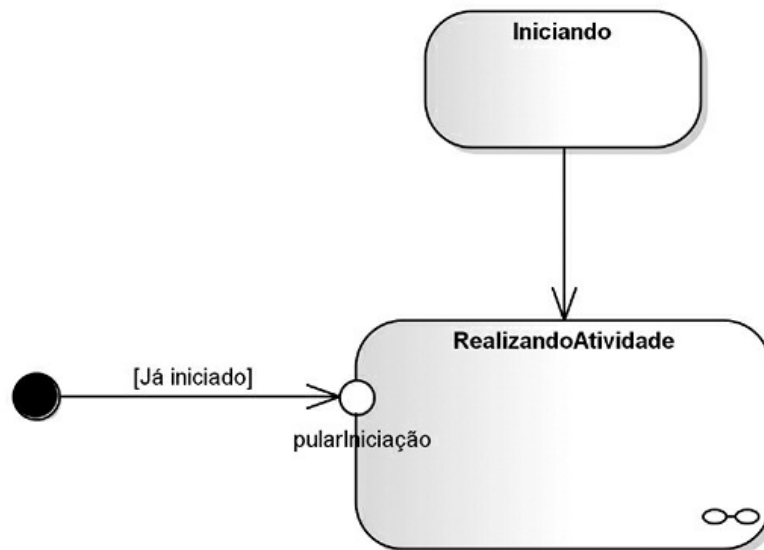
Figura 9.16 – Pseudoestado de Junção – Processo de Abertura de Conta.

Aqui, apresentamos o diagrama de máquina de estados equivalente ao processo de abertura de conta, que iniciamos a modelar nos exemplos anteriores. Depois de realizar o teste para determinar se é necessário efetuar manutenção no cadastro de clientes, observe que o fluxo foi dividido em dois, sendo novamente unido por um pseudoestado de junção, identificado aqui por uma nota. Após a união dos fluxos, gera-se uma transição para o estado **AbrindoConta**, em que é executado o método **abrirConta** e, depois de sua conclusão, gera-se uma transição para registrar o depósito inicial. Observe que essa transição aponta para um estado de submáquina em que estão detalhados os estados do processo de **Realizar Depósito**. Após a conclusão desse estado de submáquina, o processo é encerrado.

## 9.17 Pseudoestado de Ponto de Entrada e Pseudoestado de Ponto de Saída

Os pseudoestados de ponto de entrada e de ponto de saída são utilizados

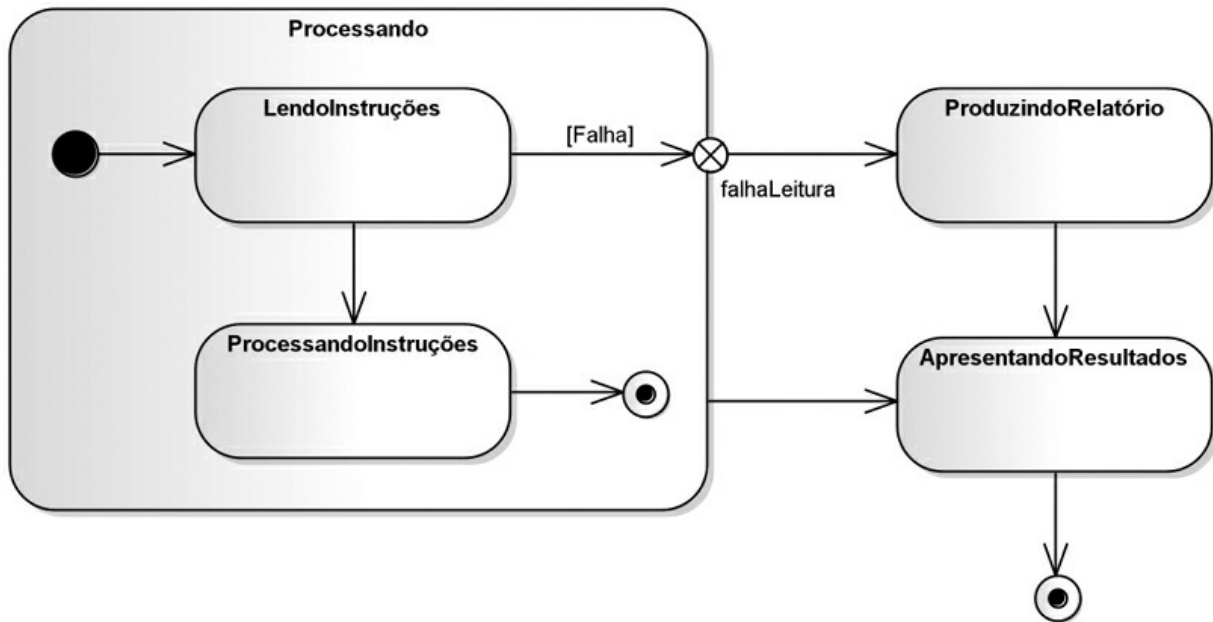
com estados de submáquina ou estados compostos. Demonstram pontos de entrada e saída que serão somente usados em casos excepcionais. O estado de entrada demonstra um caminho alternativo e é representado por um círculo vazio na borda do estado de submáquina. A figura 9.17 apresenta um exemplo de pseudoestado de ponto de entrada.



*Figura 9.17 – Pseudoestado de Ponto de Entrada.*

Nesse exemplo, o processo normal executa, primeiro, o estado denominado **Iniciando** para, depois, executar a atividade representada pelo estado de submáquina. No entanto, pode ser que não seja necessário executar o estado preliminar. Dessa forma, foi inserido no estado de submáquina um pseudoestado de entrada, chamado **pularIniciação**, por meio do qual se pode executar a atividade sem passar pela primeira etapa.

Em relação ao estado de saída, este normalmente demonstra uma exceção que causa o cancelamento do fluxo normal estado. É representado por um círculo com um X. A figura 9.18 apresenta um exemplo de pseudoestado de ponto de saída.



*Figura 9.18 – Pseudoestado de Ponto de Saída.*

Aqui, representamos um estado composto responsável por ler e processar instruções. Após sua conclusão, passa-se ao estado **ApresentandoResultados**. Porém, durante a leitura das instruções, pode ocorrer uma exceção, denotando falha de leitura das instruções. Essa exceção interrompe o processamento das instruções e gera um relatório de erros. Isso é representado por um pseudoestado de ponto de saída chamado de **falhaLeitura**.

Esses estados normalmente representam exceções, motivo pelo qual os exemplos apresentam também transições normais que não necessitam utilizar esses pseudoestados nem as entradas e saídas alternativas por eles representados.

## 9.18 Pseudoestado de Término

Força o término da execução de uma máquina de estados, em razão, por exemplo, da ocorrência de uma exceção. É representado por um “X”, conforme demonstra a figura 9.19.

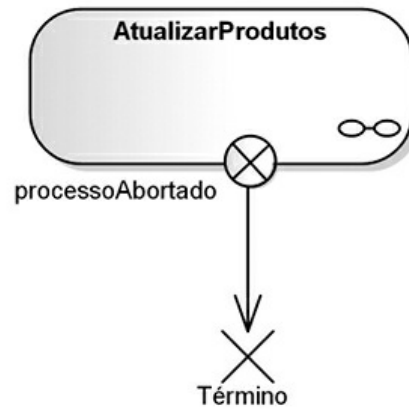


Figura 9.19 – Pseudoestado de Término.

### 9.19 Exemplo de Diagrama de Máquina de Estados – Emitir Extrato

Nesta seção, apresentaremos os estados relativos ao caso de uso **Emitir Extrato** do sistema de controle bancário que vimos modelando ao longo do livro, como pode ser visto na figura 9.20.

O processo se inicia quando a transição contendo o número da conta a consultar é disparada, produzindo o estado **ConsultandoConta**, no qual é executado o método `consultarConta`. O estado seguinte, **RecebendoSenha**, é um estado estático que aguarda que uma senha seja informada. Quando a senha é recebida, é produzido o estado **ValidandoSenha** cujo objetivo é determinar se a senha informada corresponde à senha da conta consultada. Nesse estado é executado o método `validarSenha`.

Na sequência da validação da senha, produz-se o estado **RecebendoPeríodos**, que é um estado estático em que se aguarda o fornecimento dos períodos do extrato. No momento em que os períodos são informados, passa-se a um estado composto, em que se inicia a emissão do extrato propriamente dita. Nesse estado composto, há dois subestados. O primeiro deles representa o início do extrato, em que é executado o método `emitirExtrato`. A partir desse estado, gera-se uma transição para um novo estado no qual serão consultados todos os movimentos do período relativos à conta consultada, chamado **ConsultandoMovimento**, em que é executado o método `consultarMovimento`. O leitor notará que há uma autotransição para esse

estado, enquanto ainda houver movimentos dentro dos períodos informados.

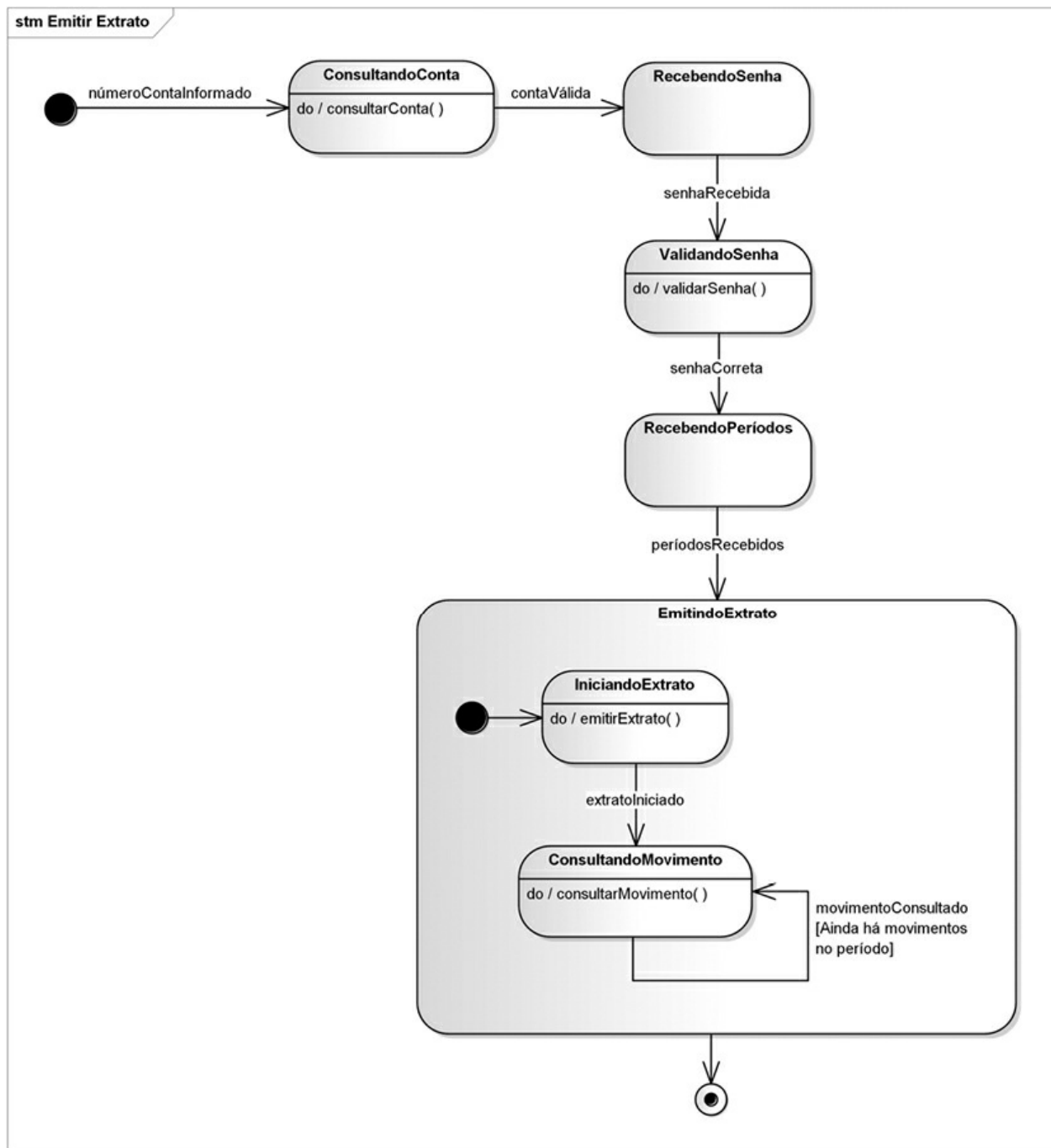


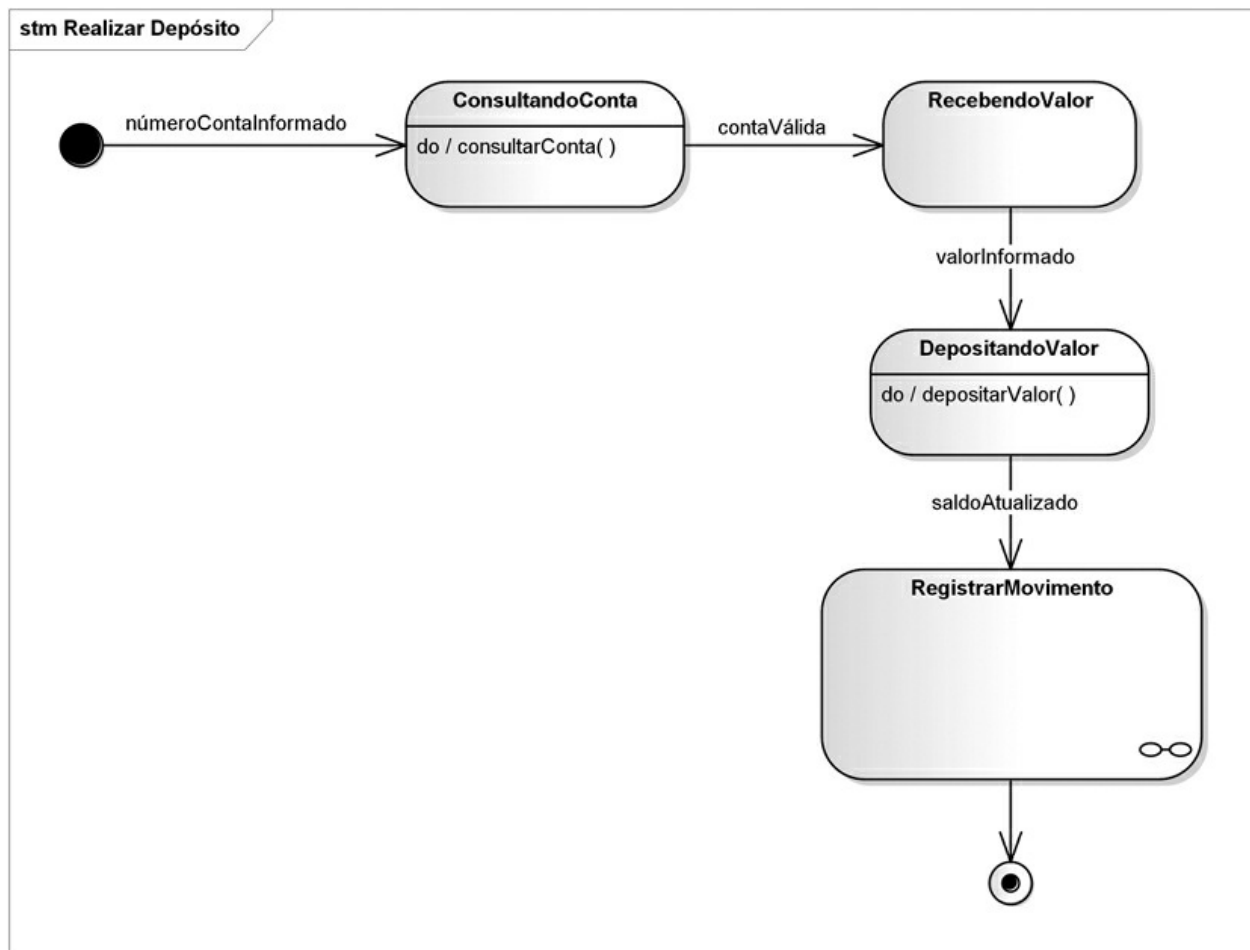
Figura 9.20 – Processo de Emissão de Extrato.

## 9.20 Exemplo de Diagrama de Máquina de Estados – Realizar Depósito

Nesta seção serão apresentados os estados relativos ao caso de uso



**Realizar Depósito** do sistema de controle bancário, por meio da figura 9.21.



*Figura 9.21 – Processo de Realizar Depósito.*

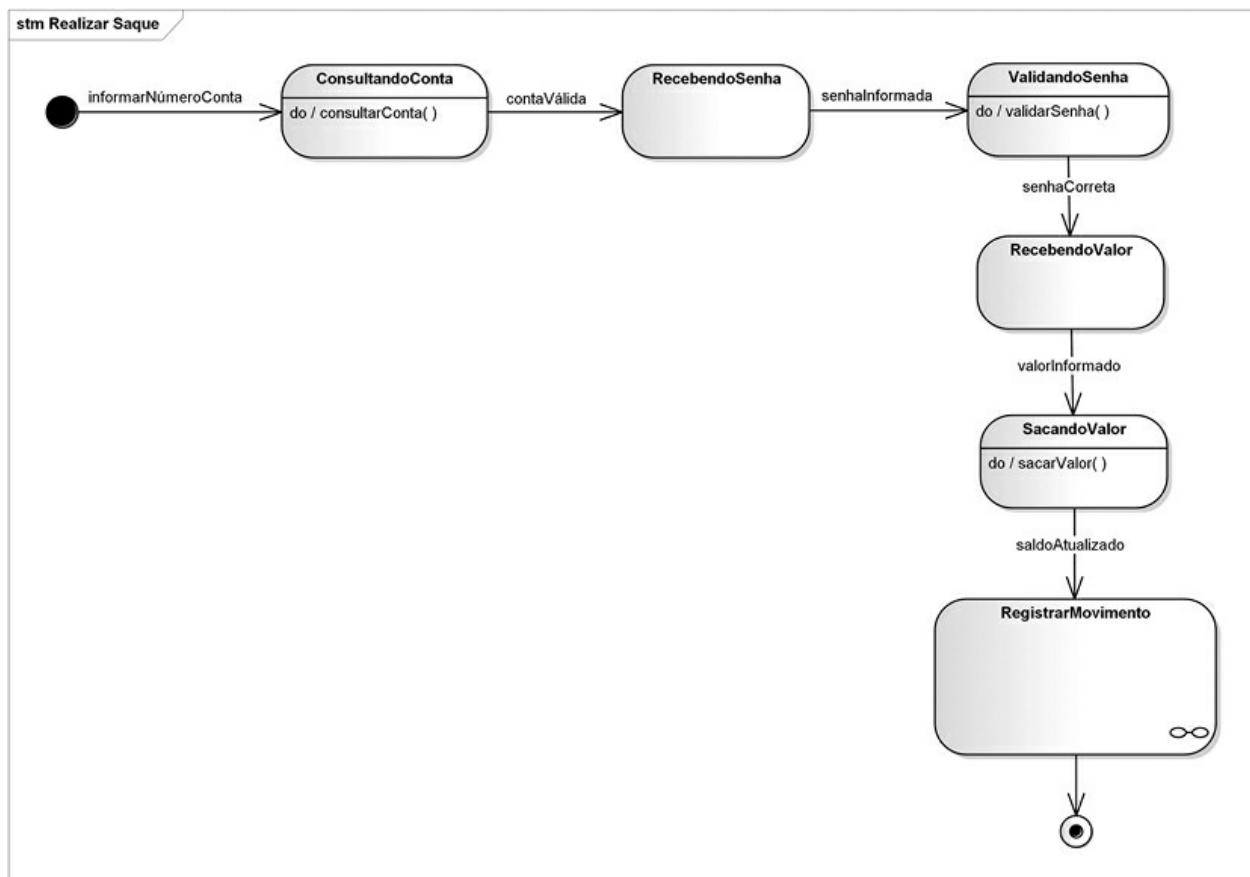
Esse diagrama se inicia quando o cliente fornece o número da conta para a qual deseja depositar um valor. Esse evento produz o estado **ConsultandoConta** em que é disparado o método **consultarConta**. Depois de a conta ter sido consultada, o evento que representa a informação de que a conta é válida gera o estado **RecebendoValor**, um estado estático em que se aguarda que o valor a depositar seja informado.

No momento em que o valor para depósito for fornecido, passa-se ao estado **DepositandoValor**, em que é executado o método **depositarValor**. A conclusão desse estado produz uma transição que leva a um estado de submáquina que representa o processo de **Registrar Movimento**. Depois de o registro do movimento ter sido concluído, essa máquina de estados é concluída.

## 9.21 Exemplo de Diagrama de Máquina de Estados – Realizar Saque

Já nesta seção, demonstraremos os estados relativos ao caso de uso **Realizar Saque**, apresentados pela figura 9.22.

Novamente, o processo se inicia com a informação do número da conta, o que gera o estado **ConsultandoConta**. Após esse estado ter sido concluído, passa-se a um estado estático que aguarda o recebimento da senha. O evento que representa o fornecimento da senha produz o estado **ValidandoSenha**, em que é executado o método **ValidarSenha**. A conclusão desse estado produz outro estado estático em que se espera que o valor a sacar seja informado.



*Figura 9.22 – Processo de Realizar Saque.*

Ao concluir-se esse estado, ou seja, quando é informado o valor desejado para saque, uma transição gera o estado **SacandoValor**, no qual é chamado o método **SacarValor**. A retirada de um valor exige o registro do movimento, por isso, após a conclusão do estado, uma transição atinge um estado de submáquina que representa o processo de **Registrar Movimento**.

e, após seu término, a máquina de estados é concluída.

## 9.22 Exemplo de Diagrama de Máquina de Estados – Encerrar Conta

Finalmente, nesta seção apresentaremos os estados pertencentes ao caso de uso **Encerrar Conta** do sistema de controle bancário, como ilustra a figura 9.23. Esse diagrama é um pouco mais complexo que os anteriores.

Uma vez que diversos processos mais simples haviam sido modelados anteriormente, lançamos mão de estados de submáquina para representá-los e deixar esse diagrama menor. Assim, o processo de encerramento de conta se inicia com o evento de solicitação de encerramento de conta, mas a transição que o representa atinge um estado de submáquina relacionado ao processo de **Emitir Saldo**, uma vez que é necessário, primeiro, saber quanto se tem na conta antes de encerrá-la. A partir desse estado de submáquina, gera-se uma transição para um pseudoestado de escolha, em que é preciso escolher entre realizar um saque ou um depósito, dependendo de se o saldo estiver positivo ou negativo.

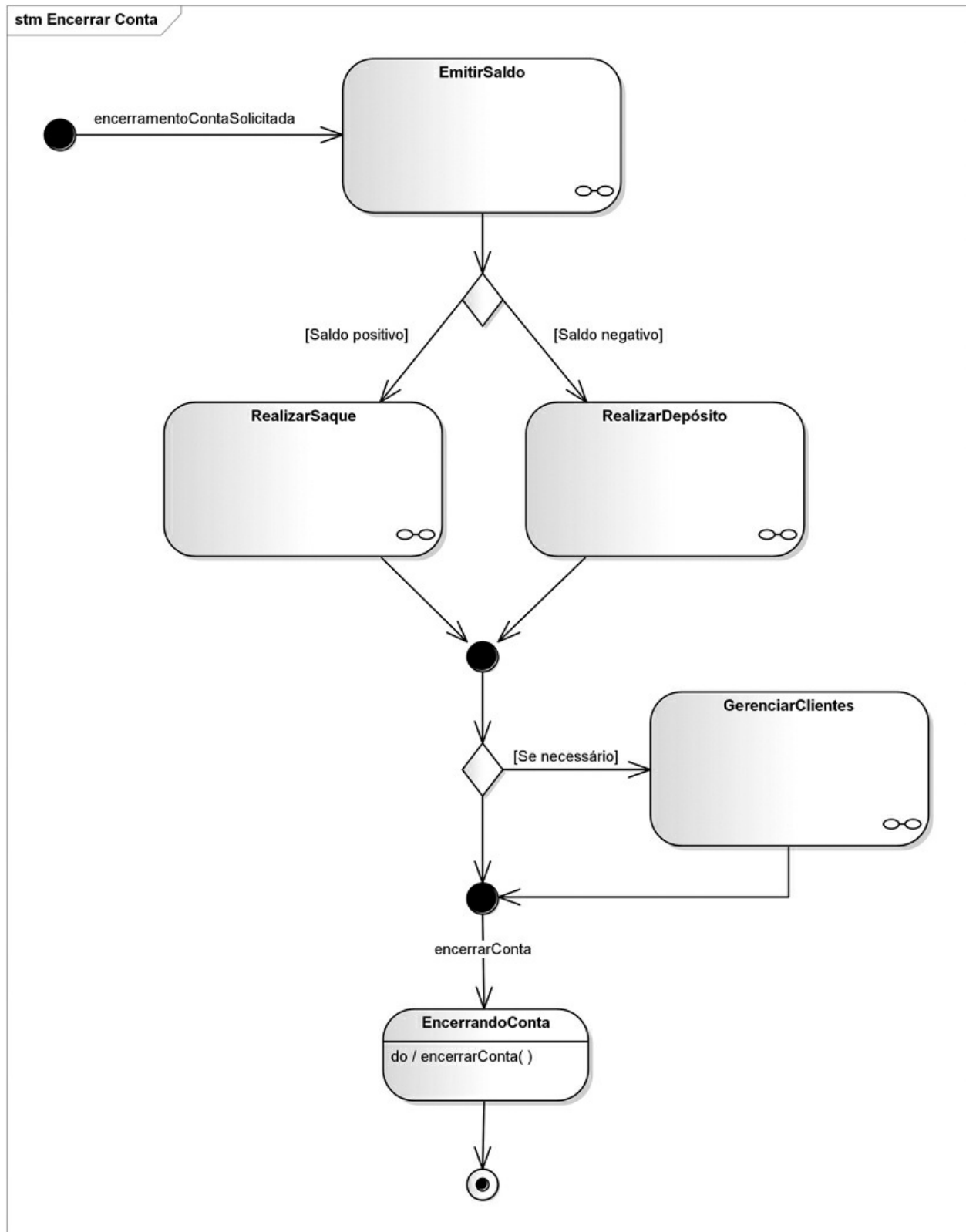


Figura 9.23 – Processo de Encerrar Conta.

Assim, se o saldo estiver positivo, gera-se uma transição para o estado de

submáquina **Realizar Saque**. Já se o saldo estiver negativo, gera-se uma transição para o estado de submáquina **Realizar Depósito**. Observe que o fluxo dividido pelo pseudoestado de escolha é unido novamente por um pseudoestado de junção e, a partir dele, é produzida uma nova transição para outro pseudoestado de escolha, em que se deve decidir se é necessário manter o cadastro do cliente, caso a conta a encerrar seja a única possuída por ele. Note ainda que novamente o fluxo dividido é unido por um novo pseudoestado de junção e, a partir deste, é produzida uma transição para o estado **Encerrando Conta**, no qual é executado o método **encerrarConta**. A conclusão desse estado também conclui o diagrama.

Poderíamos, no lugar dos estados de submáquina, ter usado estados compostos, pois estes são equivalentes às máquinas de subestados, exceto por detalharem seus subestados e não poderem ser reutilizados. Porém, isso deixaria o diagrama bem mais extenso. Poder-se-ia também detalhar todo esse processo por meio de estados simples, se o projetista assim o preferisse.

## 9.23 Exercícios Propostos

Como tem ocorrido nos capítulos anteriores, continuaremos a modelagem dos sistemas propostos pelos exercícios, enfocando agora o diagrama de máquina de estados.

### 9.23.1 Sistema de Controle de Cinema – Processo de Venda de Ingressos

Desenvolva o diagrama de máquina de estados referente ao processo de venda de ingressos para um sistema de controle de cinema, sabendo que:

- Ao selecionar a opção de venda de ingressos, o sistema deverá apresentar todas as sessões ainda não encerradas. Cada sessão deve informar o título do filme e a sala em que será apresentado.
- A partir da listagem apresentada, o funcionário deverá escolher a sessão desejada pelo cliente.
- Em resposta, o sistema deverá apresentar os assentos disponíveis da sessão.
- Com base nessa informação, o funcionário informa os assentos desejados pelo cliente e o tipo de ingresso que ele deseja, ou seja,

ingressos inteiros ou meias-entradas.

- Finalmente, o funcionário deverá gerar os ingressos referente à sessão escolhida.

### **9.23.2 Sistema de Controle de Clube Social – Processo de Pagamento de Mensalidade**

Desenvolva o diagrama de máquina de estados referente ao processo de pagamento de mensalidade para um sistema de clube social, levando em consideração os seguintes fatos:

- Primeiramente, deve-se consultar o sócio que deseja pagar mensalidades.
- Após a consulta do sócio, deve-se consultar a(s) mensalidade(s) por ele devida(s).
- Se houver alguma mensalidade em atraso, será necessário calcular os juros referentes ao atraso do pagamento.
- Em seguida, deve-se informar quais mensalidades o sócio deseja quitar.
- Finalmente, deve-se quitar a(s) mensalidade(s) informada(s).

### **9.23.3 Sistema de Locação de Veículos – Processo de Locação de Veículo**

Desenvolva o diagrama de máquina de estados referente ao processo de locação de veículo para um sistema de aluguel de veículos, considerando que:

- Ao selecionar a opção de locação de veículos, o sistema deve carregar todos os clientes registrados.
- Em seguida, o sistema deve apresentar todos os veículos disponíveis. A listagem decorrente deve mostrar a descrição do automóvel, seu modelo e marca.
- A partir dessa listagem, o funcionário deve selecionar o cliente.
- Depois de o cliente ter sido selecionado, deve-se selecionar o automóvel.
- Finalmente, depois de selecionar o veículo, o funcionário poderá mandar gerar a locação do automóvel selecionado.

### **9.23.4 Sistema para Controle de Leilão Via Internet – Processo de Realizar Leilão**

Desenvolva o diagrama de máquina de estados referente ao processo de realizar leilão para um sistema de controle de leilão via internet, de acordo com os seguintes requisitos:

- Ao receber a solicitação do serviço de realizar leilões, o sistema deve apresentar todos os leilões ainda não encerrados.
- A partir dessa listagem, o leiloeiro deve selecionar qual leilão deseja iniciar.
- No momento que um leilão for escolhido para ser iniciado, o sistema precisa carregar todos os itens a serem leiloados nele.
- A partir da listagem dos itens a serem leiloados, o leiloeiro deve escolher um item a leiloar e anunciá-lo.
- Se houver algum lance para o item anunciado, o sistema deve anunciá-lo e, em seguida, registrá-lo.
- Existe um tempo máximo de espera para que haja lances. Enquanto esse tempo não for atingido, o item permanecerá sendo anunciado.
- Quando o tempo máximo de espera por um lance for atingido, o processo deverá verificar se houve ofertas para o item, caso em que se deve anunciar o participante que ofereceu o maior lance como vencedor. Caso contrário, deve-se simplesmente encerrar o anúncio do item.
- Depois de ter sido encerrado o anúncio de um item, deve-se verificar se ainda há itens a anunciar, caso em que o processo passa a anunciar o novo item. Caso contrário, o leilão deve ser encerrado.

### **9.23.5 Sistema de Controle de Hotelaria – Processo de Pagamento de Diárias**

Desenvolva o diagrama de máquina de estados referente ao processo de pagamento de diárias para um sistema de controle de hotelaria, de acordo com as seguintes definições:

- No momento em que o hóspede informa o número do quarto para quitar as diárias, o sistema deve consultar o hóspede e todas as diárias

devidas pelo aluguel do quarto, apresentando-as ao funcionário.

- A partir dessa listagem, deve-se quitar as diárias apresentadas.
- Se tiver havido a solicitação de qualquer serviço no período em que o quarto estava ocupado, estes devem ser quitados também.
- Isso também ocorre se houver quaisquer consumos de frigobar, sendo necessário também os quitar.

### **9.23.6 Sistema de Controle de Imobiliária – Processo de Venda de Imóvel**

Desenvolva o diagrama de máquina de estados referente ao processo de venda de imóvel para um sistema de controle de imobiliária, sabendo que:

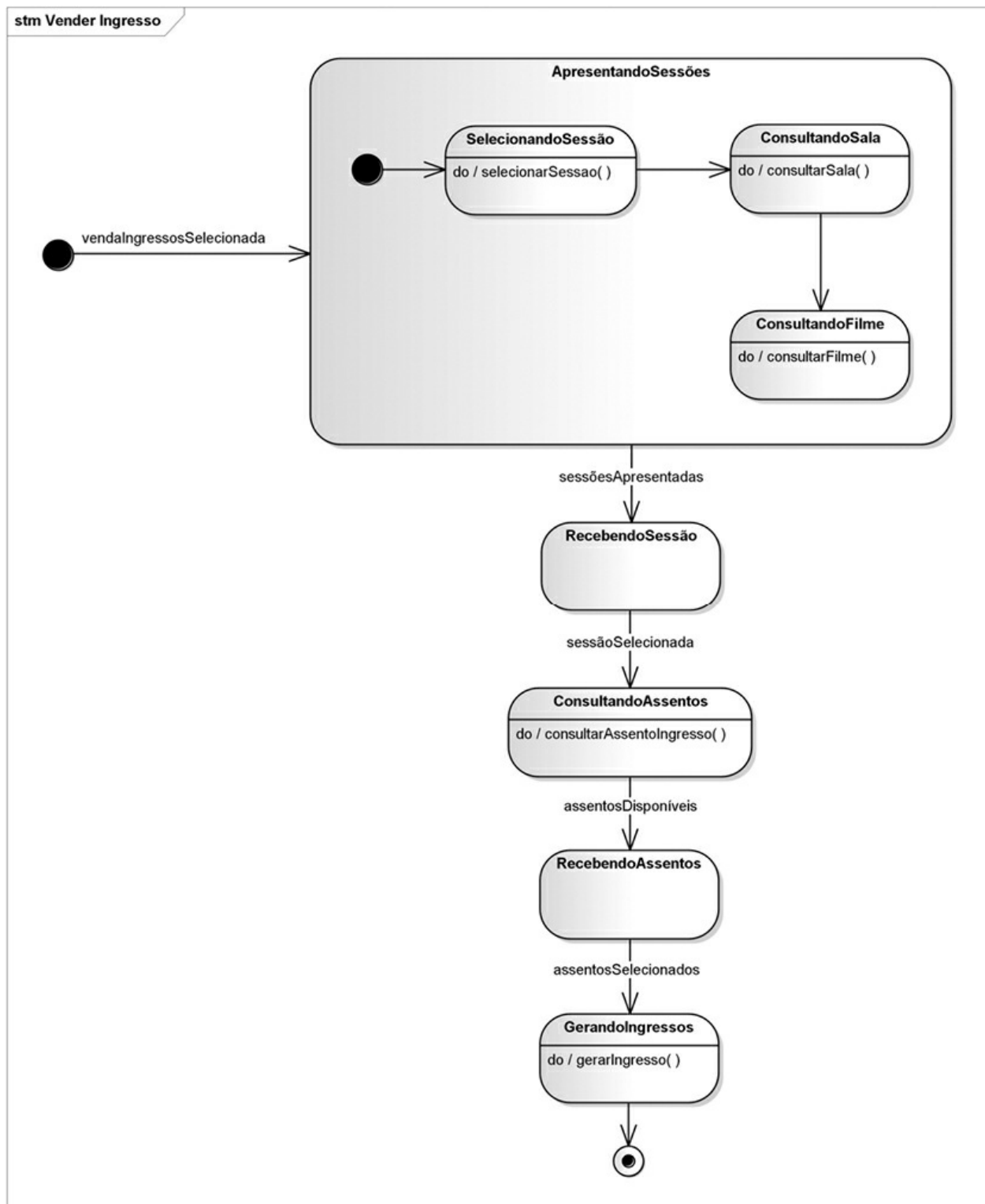
- Primeiramente, o corretor deve selecionar o tipo de imóvel que será vendido.
- Em seguida, o sistema deve listar todas os imóveis do tipo selecionado, bem como todas as pessoas registradas no sistema.
- Caso o cliente não esteja registrado, deve-se, então, registrá-lo.
- A seguir, deve-se selecionar o imóvel vendido, o cliente que o comprou e todas as taxas pagas na transação.
- Finalmente, deve-se calcular as comissões da imobiliária e do corretor, registrar a compra e todas as taxas pagas.

## **9.24 Solução dos Exercícios**

### **9.24.1 Sistema de Controle de Cinema – Processo de Venda de Ingressos**

A figura 9.24 apresenta a solução para esse exercício. A seguir, explicaremos cada um de seus estados.





*Figura 9.24 – Processo de Venda de Ingressos.*

Esse processo se inicia com o funcionário escolhendo a opção de venda de ingressos. Essa escolha gera uma transição para o estado composto

**ApresentandoSessões**, que contém três subestados. No primeiro, é selecionada uma sessão ainda não encerrada, em que é executado o método **selecionarSessao**. O subestado seguinte consulta a sala onde ocorrerá a sessão, chamando o método **consultarSala**, e o terceiro subestado consulta o filme que será apresentado, executando o método **consultarFilme**.

Ao finalizar a consulta de sessões disponíveis é gerada uma transição para um estado de espera (ou estático), em que o funcionário deverá escolher uma sessão. A escolha de uma sessão causa a transição para o estado **ConsultandoAssentos** em que todos os assentos disponíveis são apresentados. Ao concluir esse estado, passa-se ao estado estático **RecebendoAssentos**, em que se aguarda que os assentos desejados sejam escolhidos.

Quando os assentos forem escolhidos, passa-se ao último estado, **GerandoIngressos**, no qual será disparado o método **gerarIngresso** e, após a conclusão desse estado, a máquina de estados será encerrada.

Poder-se-ia argumentar que deveriam ter sido representadas autotransições nos estados descritos no plural, como **ConsultandoAssentos** ou **GerandoIngressos**, porém não consideramos necessário deixar isso explícito, uma vez que são estados simples e bastante autoexplicativos. Todavia, não estaria incorreto representar autotransições nesses estados para destacar que estes poderiam se repetir várias vezes.

### 9.24.2 Sistema de Controle de Clube Social – Processo de Pagamento de Mensalidade

A seguir, por meio da figura 9.25, apresentamos a solução para esse exercício.

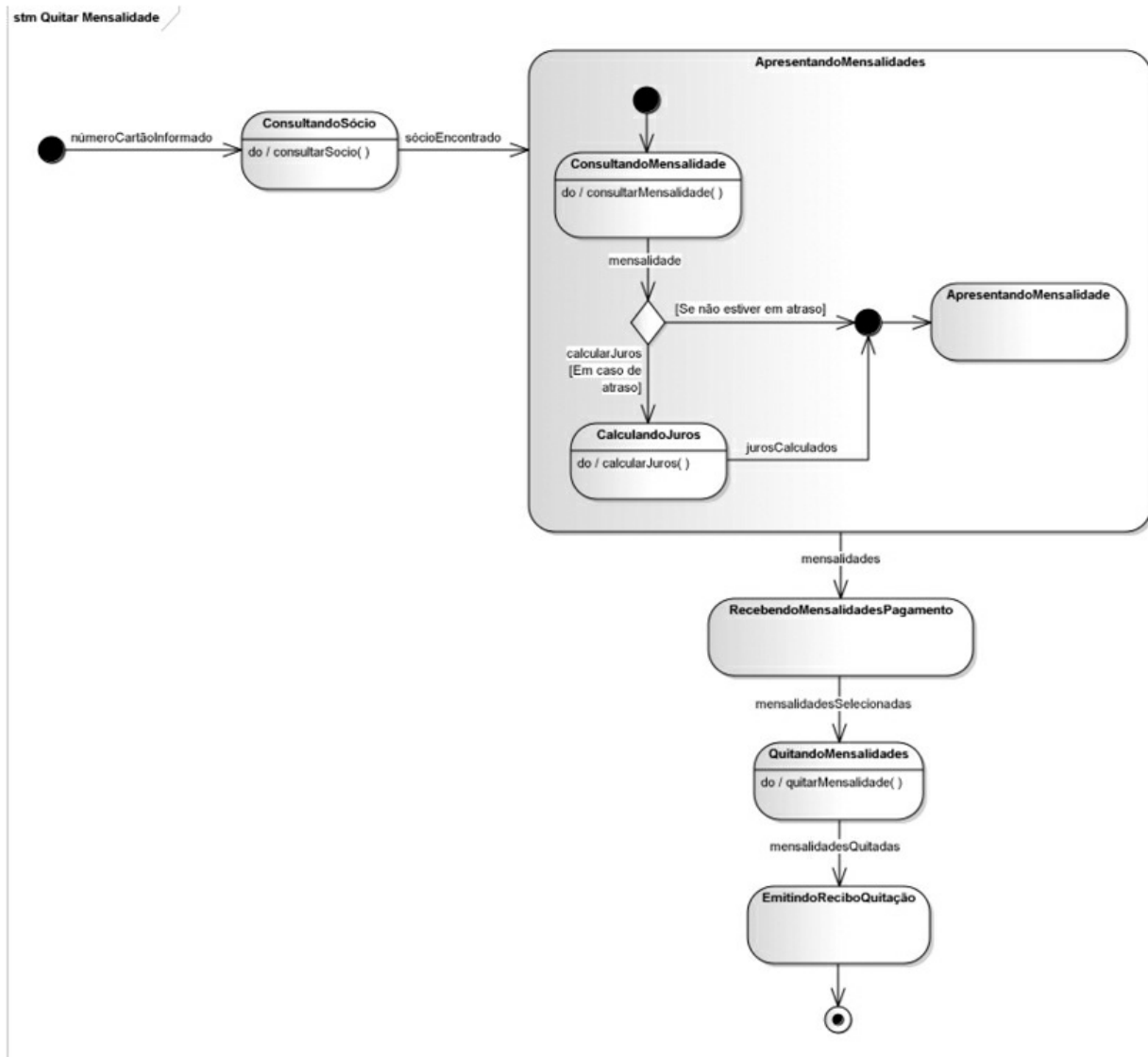


Figura 9.25 – Processo de Pagamento de Mensalidade.

Esse processo se inicia com o sócio fornecendo o número de seu cartão, o que gera o estado **ConsultandoSócio** e a execução do método **consultarSocio**. Depois dessa consulta, uma transição passa ao estado composto **ApresentandoMensalidades**. Esse estado possui três subestados, **ConsultandoMensalidade**, **CalculandoJuros** e **ApresentandoMensalidade**.

No primeiro subestado é consultada uma mensalidade específica e executado o método **consultarMensalidade**. Após a mensalidade ser consultada, uma nova transição atinge um pseudoestado de escolha em que deverá ser escolhido um entre dois fluxos possíveis. Caso o sócio não tenha mensalidades em atraso, passa-se ao estado

**ApresentandoMensalidade**, que simplesmente apresenta as informações da mensalidade a ser paga. Caso a mensalidade esteja em atraso, então se gera o estado **CalculandoJuros**, em que é disparado o método **calcularJuros** e, após o término desse método, passa-se ao estado **ApresentandoMensalidade** já explicado. Observe que o fluxo dividido é unido por um pseudoestado de junção.

Após as mensalidades terem sido apresentadas, passa-se a um estado estático em que se espera que o sócio escolha quais mensalidades deseja pagar. Essa escolha gera uma nova transição, que produz o estado **QuitandoMensalidades**, em que é executado o método **quitarMensalidade**. O término desse estado se caracteriza por uma transição para o estado **EmitindoReciboQuitação**, após o que a máquina de estados se encerra.

### 9.24.3 Sistema de Locação de Veículos – Processo de Locação de Veículo

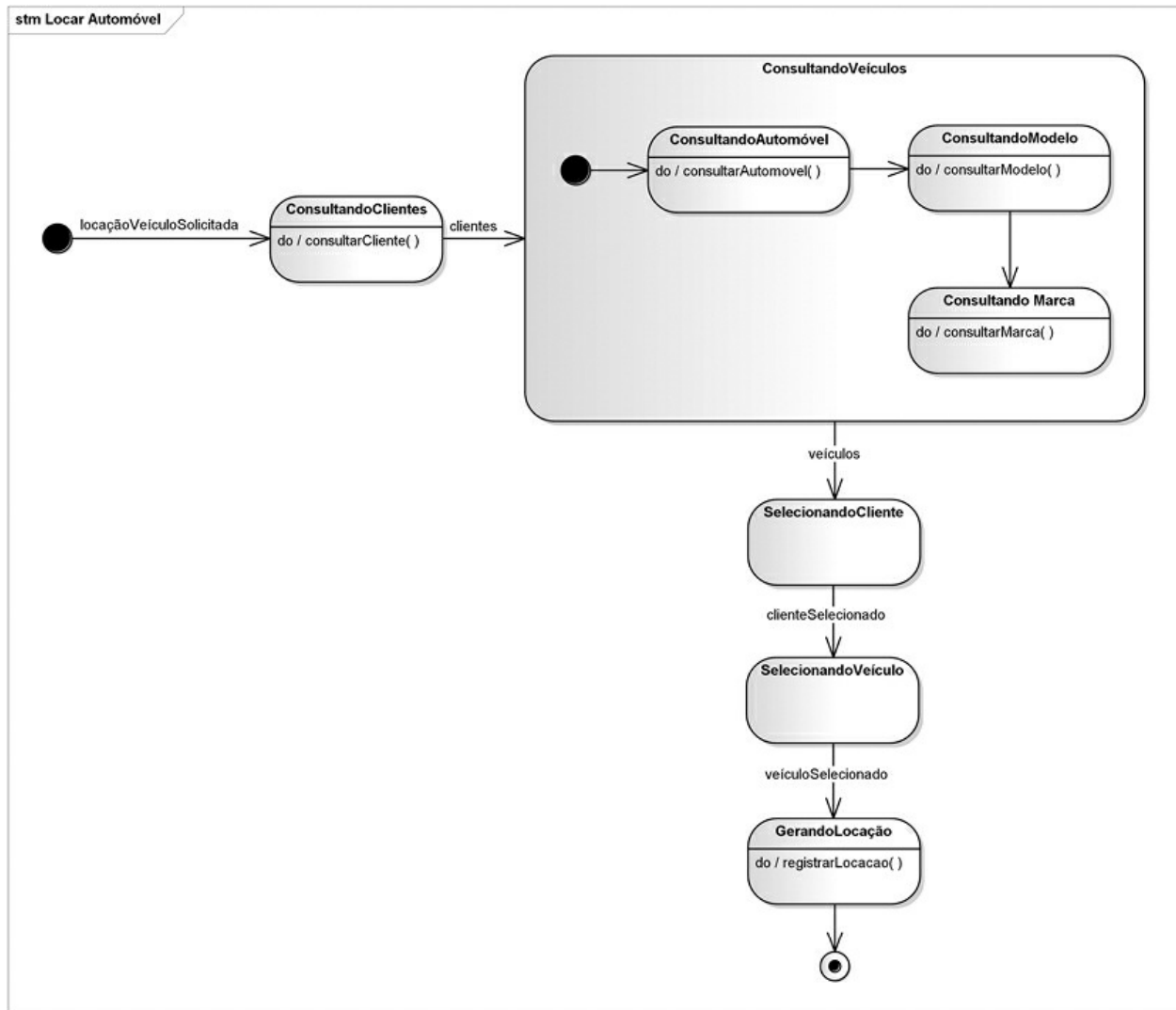
Apresentaremos aqui os estados da solução deste exercício, ilustrado pela figura 9.26.

Esse processo inicia-se com o funcionário solicitando o serviço de locação de veículos, o que causa uma transição para o estado **ConsultandoClientes**, que carrega os clientes registrados por meio da execução do método **consultarClientes**. Poder-se-ia acrescentar uma autotransição a esse estado, posto que pode se repetir várias vezes enquanto houver clientes a consultar.

Após a conclusão desse estado, passa-se ao estado composto **ConsultandoVeículos**, para carregar os veículos disponíveis. Esse estado composto apresenta três subestados. No primeiro, é consultado um automóvel e executado o método **consultarAutomovel**, no segundo é consultado o modelo do automóvel pesquisado no subestado anterior, em que é executado o método **consultarModelo** e, finalmente, no terceiro subestado, é consultada a marca do automóvel e executado o método **consultandoMarca**.

Ao ser finalizado, esse estado composto gera uma transição contendo os veículos disponíveis e passa-se ao estado **SelecionandoCliente**, em que o processo aguarda que o funcionário escolha o cliente interessado na locação. A escolha de um cliente gera uma transição para o estado

**SelecionandoVeículo**, em que o processo aguarda que seja selecionado um dos automóveis disponíveis. Por sua vez, a seleção de um automóvel gera uma transição para o estado **GerandoLocação**, em que o veículo é locado por meio da execução do método **registrarLocacao**.



*Figura 9.26 – Processo de Locação de Veículo.*

#### 9.24.4 Sistema para Controle de Leilão Via Internet – Processo de Realizar Leilão

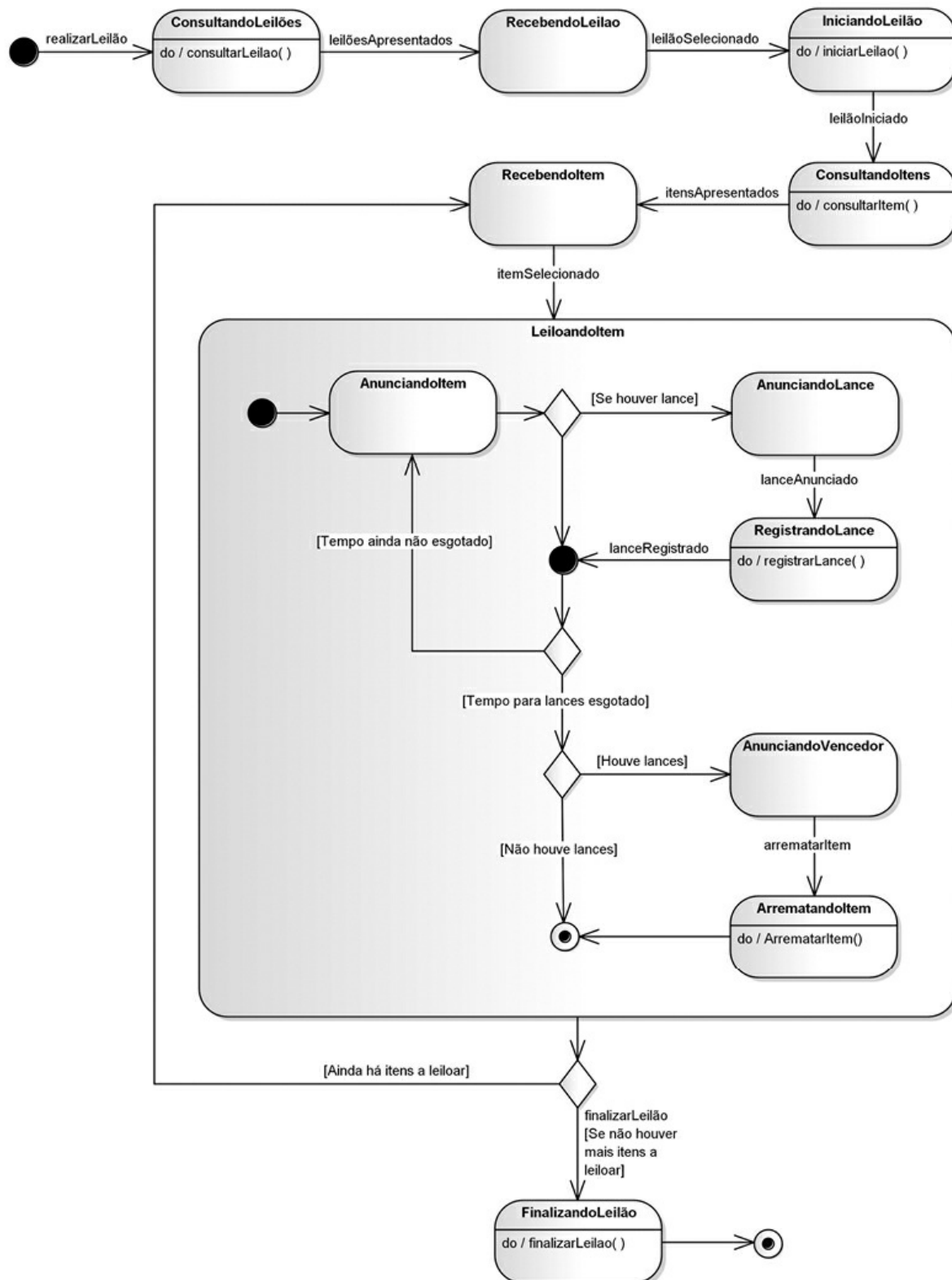
Esse processo inicia-se com o leiloeiro solicitando a execução do serviço de realizar leilões, como pode ser observado na figura 9.27. Isso causa uma transição que leva ao estado **ConsultandoLeilões**, que tem por função apresentar os leilões ainda não encerrados, executando o método

**consultarLeilao**. A partir dessa listagem, o leiloeiro deve selecionar qual leilão deseja iniciar, o que gera o estado **RecebendoLeilao**.

No momento em que um leilão é selecionado, passa-se ao estado **IniciandoLeilao**, em que é chamado o método **iniciarLeilao** e, após a conclusão desse estado, é gerada uma transição para o estado **ConsultandoItens**, cujo objetivo é carregar os itens a serem leiloados, no qual se executa o método **consultarItem**.

A partir da listagem dos itens a serem leiloados, o leiloeiro deve escolher um item a leiloar, o que é representado pelo estado **RecebendoItem**. A seleção de um item leva a um estado composto chamado **LeiloandoItem**. O primeiro subestado desse estado composto representa o anúncio do item a ser leilado. Durante esse estado, a página do leilão será atualizada para mostrar o referido item. A partir desse subestado, passa-se a um pseudoestado de escolha, no qual deve haver um teste. Se houver algum lance para o item anunciado, então se passa ao subestado que representa o anúncio dessa oferta e, em seguida, ao subestado que representa o registro dela, em que se executa o método **registrarLance**. Observe que o fluxo dividido pelo pseudoestado de escolha é unido novamente por um pseudoestado de junção.

# stm Realizar Leilão



*Figura 9.27 – Processo de Realizar Leilão.*

O pseudoestado de junção gera uma transição para um segundo pseudoestado de escolha, em que é preciso decidir se o tempo máximo de espera para que haja lances para o item foi atingido ou não. Se o tempo máximo não foi atingido, o processo volta ao estado em que o item está sendo anunciado e continua aguardando lances. Caso contrário, passa-se a um terceiro pseudoestado de escolha, em que se deve verificar se houve ofertas para o item, caso em que se gera uma transição para o subestado **AnunciandoVencedor** e, em seguida, para o subestado **ArrematandoItem**, em que é executado o método **arrematarItem**, encerrando-se os subestados do estado composto. Se não tiver ocorrido nenhuma oferta, o estado composto será simplesmente encerrado.

A conclusão do estado composto gera duas possíveis transições. A primeira verifica se ainda há itens a anunciar, caso em que o processo volta ao estado **RecebendoItem**, o que levará a um novo anúncio de item, como já foi explicado. Caso não haja mais itens a anunciar, gera-se uma transição para o último estado, no qual o leilão é encerrado, por meio do disparo do método **finalizarLeilao**, e a máquina de estados é concluída.

#### **9.24.5 Sistema de Controle de Hotelaria – Processo de Pagamento de Diárias**

A solução para esse exercício está representada na figura 9.28, cujos estados serão explicados a seguir.

Esse processo se inicia quando o hóspede informa o número do quarto do qual deseja quitar as diárias, o que gera uma transição para o estado composto **SelecionandoDiárias**. Nesse estado, existem quatro subestados. O primeiro é responsável por consultar o quarto informado, onde será disparado o método **selecionarDiarias**. Em seguida, passa-se ao segundo subestado, no qual é executado o método **consultarDiariasLocacao**, para fazer a ligação entre o quarto e o hóspede que o aluga, bem como para recuperar o período de locação. Depois, passa-se ao terceiro subestado, responsável por consultar o hóspede que está alugando o quarto, no qual é executado o método **consultarHospede**. Finalmente, passa-se para o último subestado, responsável por consultar as diárias devidas, disparando o método **consultarDiaria**.



O término desse estado composto gera uma transição para o estado **RecebendoDiariasParaPagamento**, em que são informadas quais diárias o hóspede quitará. Isto gera o estado **QuitandoDiárias**, no qual é executado o método **quitarDiaria**. A partir desse estado, uma transição encontra um pseudoestado de escolha, em que se deve testar se houve a solicitação de algum serviço no período em que o quarto estava ocupado, caso em que é gerada uma transição para um estado de submáquina referente ao processo de **Quitar Serviços**.

O fluxo dividido pelo pseudoestado de escolha é reunido por um pseudoestado de junção que gera uma transição para um segundo pseudoestado de escolha, em que é necessário testar se houve algum consumo de frigobar, caso em que é gerada uma transição para um segundo estado de submáquina que representa o processo de **Quitar Consumo**. O fluxo dividido é reunido por um segundo pseudoestado de junção e a máquina de estados é encerrada.

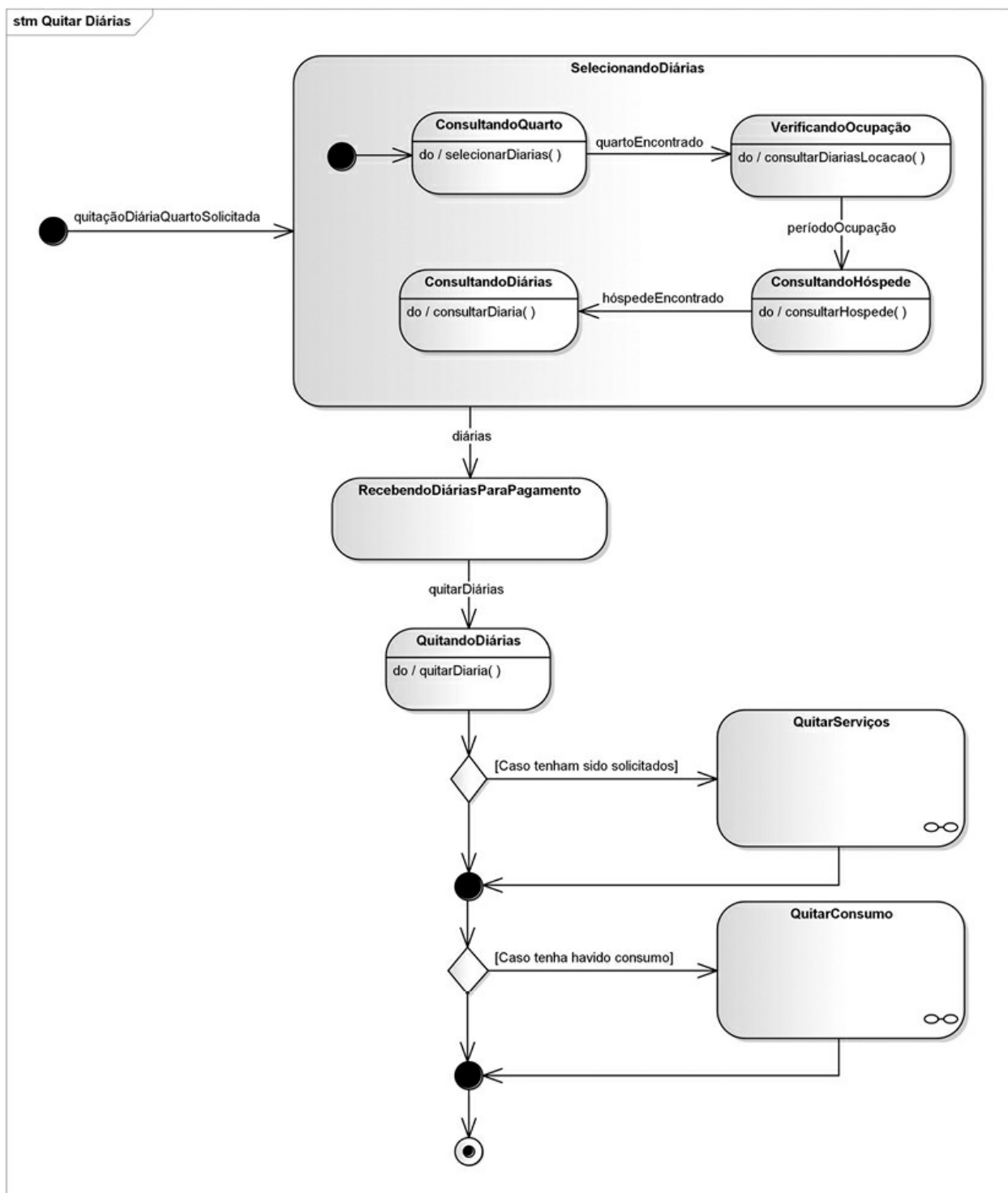


Figura 9.28 – Processo de Pagamento de Diárias.

## 9.24.6 Sistema de Controle de Imobiliária – Processo de Venda de Imóvel

A figura 9.29 ilustra a solução para esse exercício.

Após o evento inicial, produz-se o estado **RecebendoTipoImóvel**, em que se aguarda que o tipo de imóvel a ser vendido seja informado. Quando a seleção do tipo de imóvel ocorre, gera-se o estado **ListandoImóveis**, em que são apresentados todos os imóveis do tipo selecionado. Depois, passa-se ao estado **ListandoPessoas**, que apresenta todas as pessoas registradas no sistema.

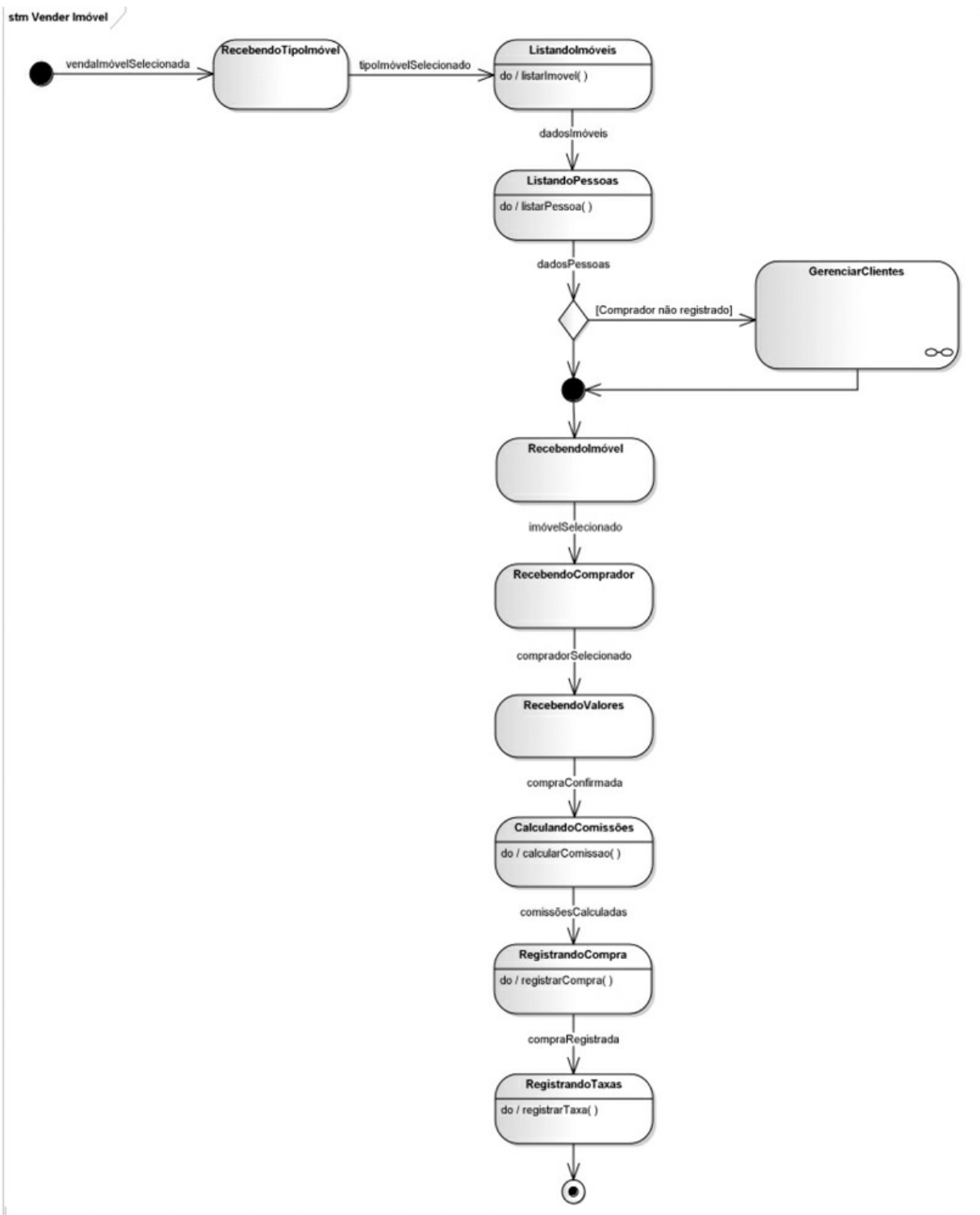


Figura 9.29 – Processo de Venda de Imóveis.

A partir dessa listagem é realizado um teste para determinar se o comprador se encontra na lista, caso contrário é executado o processo de **GerenciarClientes**, para cadastrá-lo. Esse processo é representado aqui por

uma máquina de subestados.

Depois disso, passa-se ao estado **RecebendoImóvel**, em que se aguarda que o imóvel desejado seja selecionado. Depois, passa-se ao estado em que se aguarda que o comprador seja escolhido e, então, ao estado em que os valores da transação são informados.

Nos estados seguintes são calculadas as comissões da imobiliária e do corretor, registrada a compra e, finalmente, registradas as taxas pagas, o que finaliza a máquina de estados.